COMSATS University Islamabad (CUI)

Project Name

By

Abdullah Akram **CIIT/FA17-BSE-005/ISB**

Ali Zain **CIIT/FA17-BSE-016/ISB**

Supervisor

Majid Iqbal Khan

Co-Supervisor

Zaheer ul Hussain Sani

Bachelor of Science in Software Engineering (2017-2021)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

COMSATS University Islamabad (CUI)

LightHouse II

A project presented to

COMSATS University Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Software Engineering (2017-2021)

By

Abdullah Akram CIIT/FA17-BSE-005/ISB

Ali Zain CIIT/FA17-BSE-016/ISB

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Abdullah Akram

Abdullah

Ali Zain

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(SE) "LightHouse-II" was developed by **ABDULLAH AKRAM (CIIT/FA17-BSE-005)** and **ALI ZAIN (CIIT/FA17-BSE-016)** under the supervision of "Dr. Majid Iqbal" and co-supervised by "Mr Zaheer Sani" and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

Supervisor

Co-Supervisor

External Examiner

Head of Department

(Department of Computer Science)

Executive Summary

The Student Assessment Platform is a web-based application for aspiring programmers here in Comsats University. Each user will be preregistered by the department. The registered students will be able analyzed by the system throughout the semester. Through AI we will study the students and then differentiate which student is at risk and which student is excelling at a remarkable speed. The system would guide the students on how to code. Challenges will be set, and the students will be ranked weekly based on how good they were in completing the challenge. Lecturers will be given a weekly summary of their students and the system will guide which students require more attention. The system would help the students to code, guide them to tackle errors and provide necessary resources to help clear concepts as per student need.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor "Dr. Majid Iqbal Khan" and our Co-supervisor "Mr. Zaheer Ul Hasan Sani". Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Abdullah Akram

Ali Zain

Abbreviations

SRS	Software Requirement Specification
IDE	Integrated Development Environment
SDS	Software Design Specification

Table of Contents

List of Figures

1 Introduction

1.1 Brief Overview

The student assessment portal built using ReactJs, NodeJs, SQL and is designed to assist the lecturers in their teaching programming languages to students. This portal will currently cover the JAVA language as its main programming language. The portal will have an admin dashboard that will allow a user with admin privileges to control the entire system, then there will be a lecturer dashboard that will control and manipulate data relating to students and their tasks and finally a student dashboard that will allow them to learn and code in a friendlier environment. The portal for a student will contain an IDE where a student can code, his/her code will be checked and tested with relative test cases and upon task verification that the test case has been fulfilled only then will the task be considered as complete. Students will be marked quantitatively upon task completion. There will be however certain red alerts set that will be triggered on certain events to catch unethical means of task completion. The lecturer can choose to deduct marks of the student or reassign them a different task altogether. The goal of this assessment portal is to provide a more transparent and easier way of teaching how to code so we can distinguish between excelling students and at-risk students. That way a teacher can handle the student accordingly and by the end of the semester each student will have an equal level of understanding on how to code.

1.2 Relevance to Course Modules

Lighthouse-II has relevance to following course modules:

i. Introduction to Computer Technology
ii. Programming Fundamentals
iii. Object-Oriented Programming
iv. Data Structures and Algorithms
v. Topics in Software Engineering (Part 1)
vi. Web Technologies
vii. Software Requirement Engineering
viii. Software Design and Architecture
ix. Software Testing
x. Software Quality Engineering

1.3 Project Background

Individual assessment is a tough job. Not every lecturer has the time to individually assess which student is good at programming and which is not. Not all students are familiar with programming as they may come from different fields. So, the lecturer could never know beforehand who a quick learner is and who needs special catering to clear their concepts. The lecturer also can help a student individually to an extent as they have other matters to attend as well so they are not always readily available. We require a system that would automatically assess the students and teach them as per their skills. This would help in reducing the workload on a lecturer and within the start of the semester the lecturer would find out which student is lacking, and which student is excelling. The system will assist the lecturer in enhancing the skills of the students so that by the end of the semester each student will be on equal grounds in programming. A similar famous website is Code academy that has a similar system as to the one we are building but our system will be built using ReactJs, NodeJs, SQL. By the end of this project, we will have a firmer grip on the JavaScript and will also get familiarized with the implementation products built around it.

1.4 Methodology and Software Lifecycle for this Project

This platform will be built using the agile process model. We will increment modules into our system as we go. This process model is beneficial in this manner as it supports development of products that have changing requirements. In the worst-case scenario where any one of our modules does not work, we can rework on it and increment it back into our product without any hassle. We would not have to be concerned with formalities such as documenting each change as we code, rather we can just log the change into a single change log file and include it in our final documentation. This model suits those product inventions that are not that very common in the market or are not even in the market, in case of our product we are designing something that is not very common in the market and so with such uncertainty in what our final product will contain we would have to test many modules and after thorough testing only then we would see which module makes it to the final cut. This model supports testing more than any other model and thus it will be our choice of procedure. For our methodology we will be following a mixture of OOP and Procedural methods as NodeJS is a running environment for JavaScript and JavaScript takes the benefits of both OOP and Procedural methods to provide optimal processing at runtime.

2 Problem Definition

2.1 Problem Statement

Individual assessment is a tough job. Not every lecturer has the time to individually assess which student is good at programming and which is not. Not all students are familiar with programming as they may come from different fields. So, the lecturer could never know beforehand who a quick learner is and who needs special catering to clear their concepts. The lecturer also can help a student individually to an extent as they have other matters to attend as well so they are not always readily available. We require a system that would automatically assess the students and teach them as per their skills. This would help in reducing the workload on a lecturer and within the start of the semester the lecturer would find out which student is lacking, and which student is excelling. The system will assist the lecturer in enhancing the skills of the students so that by the end of the semester each student will be on equal grounds in programming. A similar famous website is Code academy that has a similar system as to the one we are building but our system will be built upon using ReactJs, NodeJs, SQ By the end of this project, we will have a firmer grip on the JavaScript and will also get familiarized with the implementation products built around it.

2.2 Deliverables and Development Requirements

This section would need to discuss about the deliverables and the development requirements.

2.2.1 Module 1: Administration

An admin has the highest privileges out of all the users. An admin will be able to:

- Perform crud functions on a lecturer type use.
- Perform crud functions on a student type user.
- Perform crud functions on challenges and set their cases.
- Perform crud functions of subjects.
- Can view student rankings in their relative subjects and thus can analyze what are the mean statistics of a classroom, this way the admin can also get a clearer picture of a lecturer's performance.

2.2.2 Module 2: Lecturer controls

A lecturer has the second highest privileges. A lecturer can:

- Perform crud functions on his own profile.
- Perform crud functions on a student's profile.
- Perform crud functions on lab tasks.
- Can view the rankings of his class.
- Can assign practice tasks and assess them manually.

2.2.3 Module 3: Student controls

A student has the lowest privileges and is not allowed any crud functionality to keep the integrity of the system intact. A student can:

- Code on the IDE and view resources
- Can view his rankings in the class.

2.2.4 Module 4: Rating and Feedback

In this module a lecturer can analyze some data sets and if he/she finds that the data set belongs in wrong category i.e., easy, medium, hard, then the lecturer can report this to the administrator to update the data set. Upon inspection it is up to the admin to update the data sets. Not only that but the lecturer can report other various other issues as well.

A student can also give feedback regarding the system. How it can be improved and what issues they may face during coding and if they find the ranking system to be too harsh or underdeveloped. The feedbacks are transparent, and anyone can view them. If a student has similar issue, they can upvote the issue so that I can be placed at the highest priority and can be resolved immediately.

For frequently asked questions the admin panel will pin answers to those questions so that they would not have to deal with repetitive questions.

2.2.5 Module 5: IDE

Our system will have an IDE where students will learn to code. The IDE will be the main UI for a student. Here we will support coding, result generation, error generation, match test case and see if the results match with the expected results. The IDE will have a button where a student can request for the solution and if a lecturer allows to view the solution the solution will be displayed to that user, although that would negatively affect the student's ranking. The IDE will see that a user is facing the same error again and again so it will provide relative resources to assist the user to help him/her to resolve the issue. The IDE will currently entertain the JAVA language only.

2.2.6 Module 6: Application Settings

The user can apply settings to their profiles as per their requirements. They can edit their own details as allowed and can manage their account as per their requirement.

2.2.7 Module 7: Assessment

The assessment of students will be quantitative. We will compare the students written code by testing them against the already given test cases. The software will judge if all the requirements of the questions are met or not using AI. If the tasks are found to be lacking some piece of logic then they will be notified immediately so they can adjust their code to completed the assignment properly. If the student completes the task without any issue, he will be graded by the teacher on the bases of their solution. There will be some red alerts in place for suspicious activity such as if the student completes a task before the minimum time, then the lecturer will be notified of the suspicious activity. If the lecturer allows the student can gain full marks or the lecturer can deduct marks as per his/her choice. If a student faces any issue while coding, he/she will be given similar tasks from the data sets to further clear their confusions. The more extra tasks you do the more bonus you can gain as per the teacher's choice. Each student will be ranked as per their performance by the end of the class and thus this way teacher will know which student struggled during the task and which student completed it with flying colors.

2.3 Current System

This dashboard is a complete application designed as a Web Application to run on any hardware and software platform without compromising UX or functionality. It integrates all the features which are required for lecturers/faculty to monitor their students all together on one platform. It has separate accounts for both students and lecturers which ensures they perform specialized functionality as well. Being responsive in nature ensures that cross-platform usage will be promoted, and all application instances will perform without issues.

3 Requirement Analysis

3.1 Use Cases Diagram(s)

The following use cases were drawn in MS Visio 2016. The standards of UML version 2.5 have been followed.

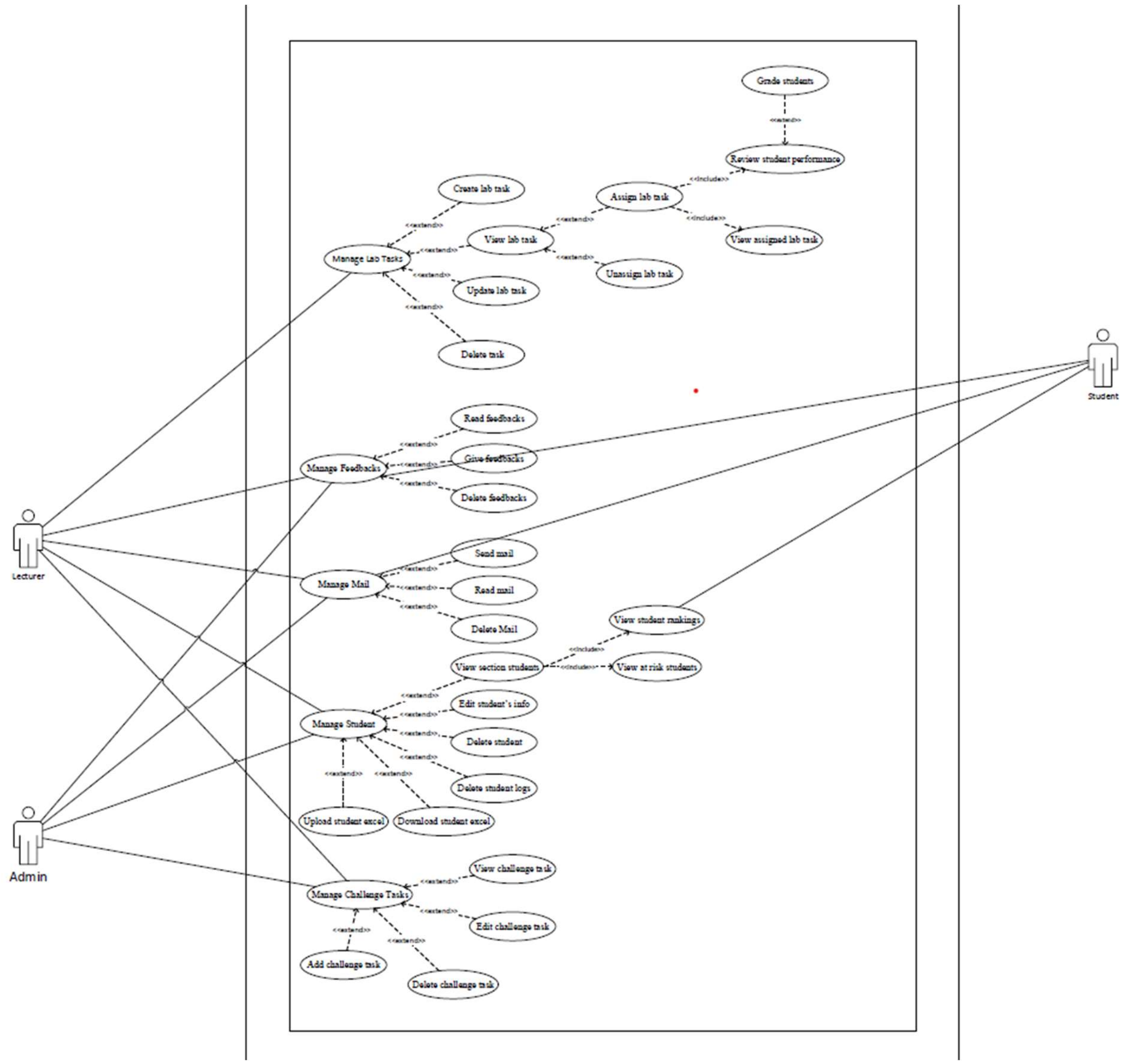

Figure 1: Use Case Diagram

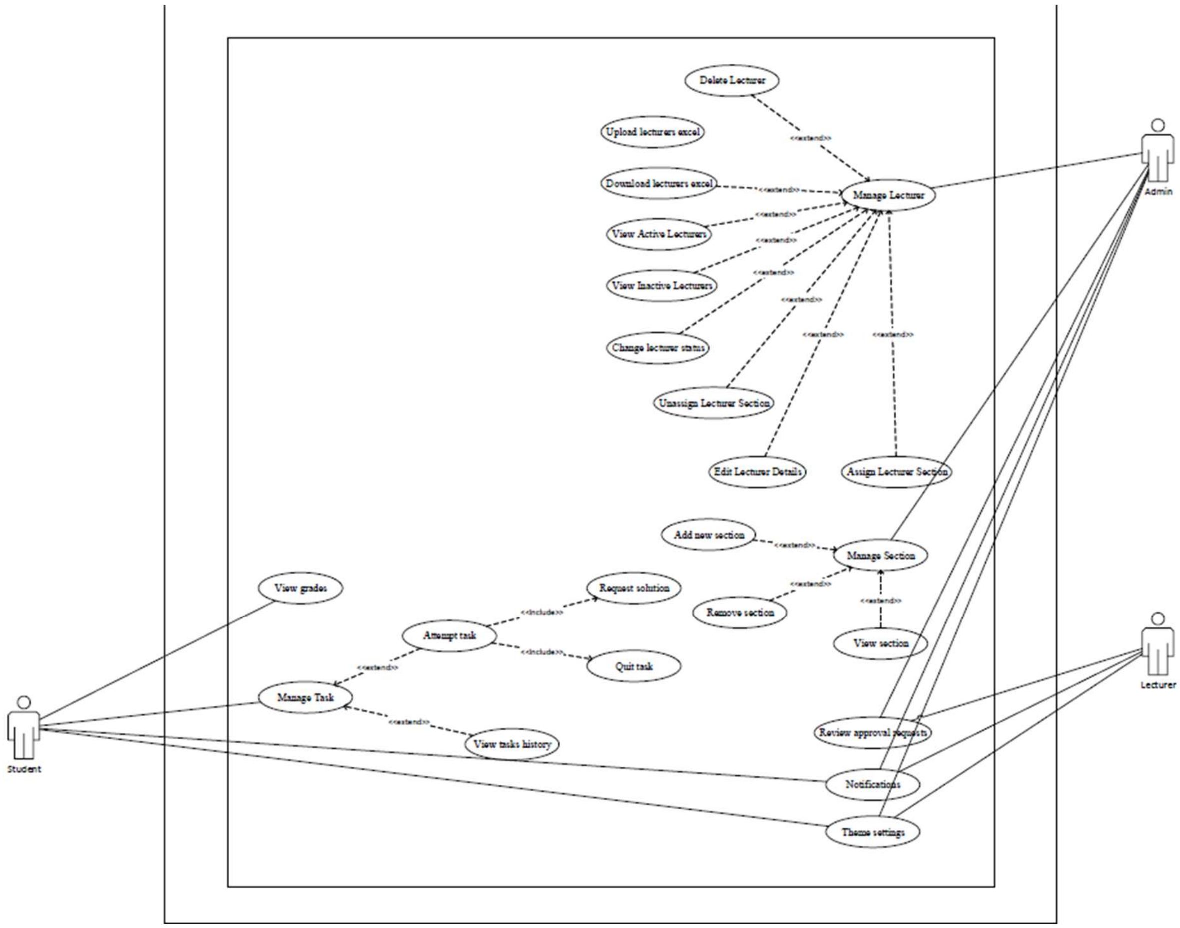

Figure 2: Use Case Diagram

3.2 Detailed Use Case

3.2.1 All Actors Perspective:

3.2.1.1 UC-1: Login

Use Case ID:	UC-1
Use Case Name:	Login
Actors:	Primary Actor: Admin, Lecturer, Student
Description:	The Actors involved will be able to login to their respective portals via a login page. Upon selecting the type of actor, they are they will be prompted to a login page designed for their specific role.
Trigger:	An actor decides to login to their respective portal to be able to perform various tasks offered by the system as per their need.
Preconditions:	PRE-1. Actor visits the website link. PRE-2. Actor has selected their role i.e., is he and Admin, Lecturer, or a student
Postconditions:	POST-1. Actor has been validated. POST-2. Actor can now view his Dashboard
Normal Flow:	1. User Visits the link to the assessment portal. 2. The assessment portal greets the user. 3. The portal asks the user of its role. 4. The user selects their role. 5. The portal then takes the user to their respective logins. 6. The user inputs their assigned username and password 7. The user is validated. 8. User session is stored in the database. 9. The user can now view their respective Dashboards
Alternative Flows: [Alternative Flow 1 – Not in Network]	7.1. The user is invalid. 7.2. Portal displays an error "Invalid username or password." 7.3. The user is returned to step 5 in normal flow. 8.1. User session has reached max limit. 8.2. User is prompted to terminate all previous sessions. 8.3. User terminates all previous sessions. 8.4. User is returned to step 9 in normal flow
Exceptions:	1. User does not have an internet connection to login. 2. The user failed to validate
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.1.2 UC-2: Logout

Use Case ID:	UC-2
Use Case Name:	Logout
Actors:	Admin, Lecturer, Student
Description:	A user can logout from their account
Trigger:	The User clicks on logout icon in the navigation
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User clicks on the logout icon
Postconditions:	POST-1. User successfully logs out.
Normal Flow:	1. The user clicks on the logout icon. 2. The user is logged out of the dashboard. 3. User session is destroyed
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.1.3 UC-3: View Student rankings

Use Case ID:	UC-3
Use Case Name:	Viewing student rankings
Actors:	Admin, Lecturer, Student
Description:	A user can view the rankings of a student within their respective sections. A Lecturer and a student can only view their own respective registered section's rankings
Trigger:	The user clicks on view rankings navigation link under the section summary page
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User is currently viewing the section summary page. PRE-3. User clicked on the view students' rankings link
Postconditions:	POST-1. The student rankings are displayed.

Normal Flow:	1. The user clicks on view sections link under the manage section navigation. 2. The user clicks on the target section. 3. The user clicks on the students ranking link. 4. The system displays the students ranking within the section.
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.1.4 UC-4: Send Mail

Use Case ID:	UC-4
Use Case Name:	Send Mail
Actors:	Admin, Lecturer, Student
Description:	A user can create a mail and can send to anyone with an account on the portal.
Trigger:	The user writes and sends a mail by pressing send on the create mail page
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User created a new mail. PRE-3. User sent the mail
Postconditions:	POST-1. The mail is successfully sent and received by both involved parties.
Normal Flow:	1. The user clicks on the send mail link under Mailing navigation. 2. The user writes some mail. 3. The user presses send to send the mail
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None

Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.1.5 UC-5: Read Mail

Use Case ID:	UC-5
Use Case Name:	Read Mail
Actors:	Admin, Lecturer, Student
Description:	A user can read their received mail
Trigger:	The user clicks on the view mail link.
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User pressed the view mail link
Postconditions:	POST-1. The user can view their mail.
Normal Flow:	1. The user clicks on the view mail link under Mailing navigation. 2. All their mail is viewed to them. 3. A user can click on any mail to view it
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.1.6 UC-6: Delete Mail

Use Case ID:	UC-6
Use Case Name:	Delete Mail
Actors:	Admin, Lecturer, Student
Description:	A user can delete their received mail
Trigger:	The user clicks on the delete mail button.
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User pressed the view delete mail button Infront of their target mail
Postconditions:	POST-1. The user successfully deleted the target mail.

Normal Flow:	1. The user clicks on the view mail link under Mailing navigation. 2. All their mail is viewed to them. 3. The user pressed the delete button Infront of their target mail. 4. The target mail is deleted.
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.1.7 UC-7: Read feedbacks

Use Case ID:	UC-7
Use Case Name:	Read Feedbacks
Actors:	Admin, Lecturer, Student
Description:	A user can read feedbacks given by other users or themselves.
Trigger:	The User clicks on feedback link under the section summary page
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User is currently viewing the section summary page. PRE-3. User clicked on the feedback link
Postconditions:	POST-1. Feedbacks by the section are being displayed on the feedback page.
Normal Flow:	1. The user clicks on the view sections link under the manage sections navigation. 2. The user clicks on the target section link. 3. The section summary page is being displayed to the user. 4. The user clicks on the feedback link. 5. The feedback page is being displayed to the user.
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.

Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.1.8 UC-8: Give feedbacks

Use Case ID:	UC-8
Use Case Name:	Give Feedbacks
Actors:	Admin, Lecturer, Student
Description:	A user can give feedbacks.
Trigger:	The User clicks on submit feedback button on the feedback page
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User is currently viewing the section summary page. PRE-3. User clicked on the feedback link. PRE-4. User input some text in the feedback page. PRE-5. User submits the feedback
Postconditions:	POST-1. The feedback is successfully submitted by the user and is now being displayed.
Normal Flow:	1. The user clicks on the view sections link under the manage sections navigation. 2. The user clicks on the target section link. 3. The section summary page is being displayed to the user. 4. The user clicks on the feedback link. 5. The feedback page is being displayed to the user. 6. The user inputs a feedback in the input box. 7. The user submits the feedback. 8. The system views the feedback on forum
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.1.9 UC-9: Delete feedbacks

Use Case ID:	UC-9
Use Case Name:	Delete Feedbacks
Actors:	Admin, Lecturer, Student
Description:	A user can delete their feedbacks. An Admin can delete user's feedback
Trigger:	The User clicks on delete feedback button under their feedback on the feedback page
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User is currently viewing the section summary page. PRE-3. User clicked on the feedback link. PRE-4. User clicks on the delete button under their feedback
Postconditions:	POST-1. The feedback is successfully deleted.
Normal Flow:	1. The user clicks on the view sections link under the manage sections navigation. 2. The user clicks on the target section link. 3. The section summary page is being displayed to the user. 4. The user clicks on the feedback link. 5. The feedback page is being displayed to the user. 6. The user clicks on the delete button under their feedback. 7. The feedback is successfully deleted
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.1.10 UC-10: Notifications

Use Case ID:	UC-10
Use Case Name:	Notifications
Actors:	Admin, Lecturer, Student
Description:	A user can view all their new notification under the notification icon
Trigger:	The user receives some notification

Preconditions:	PRE-1. User receives some notification
Postconditions:	POST-1. The user views their notification.
Normal Flow:	1. The user clicks on notification icon. 2. The user can now view their recent notifications
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.1.11 UC-11: Theme settings

Use Case ID:	UC-11
Use Case Name:	Theme settings
Actors:	Admin, Lecturer, Student
Description:	A user can set the color of the theme according to their preference
Trigger:	The user clicks on the change theme icon
Preconditions:	PRE-1. User clicks on the change theme icon
Postconditions:	POST-1. The user changed the theme according to their preference.
Normal Flow:	1. The user clicks on the change theme icon. 2. The user changed the theme according to their preference
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None

Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2 Admin Perspective:

3.2.2.1 UC-12: View Inactive Lecturers

Use Case ID:	UC-12
Use Case Name:	View Inactive Lecturers
Actors:	Admin
Description:	An Admin can view all the current Inactive lecturers list on the portal who are currently not teaching a programing related subject but have previously done so.
Trigger:	The admin clicks on the view inactive lecturer's navigation link under Manage Lecturers.
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicked on the View Inactive Lecturers Nav Link
Postconditions:	POST-1. The admin can now view of all the inactive active lecturers list. POST-2. The admin can now click on any one of the lecturers to see a summary of their performance history, edit their details, set them as active or delete them from the database.
Normal Flow:	1. The admin has validated and is logged in. 2. The admin is now viewing his dashboard. 3. The admin hovers over the drop down "Manage Lecturers." 4. Under the Manage Lecturers section the admin clicks on the link "View Inactive Lecturers" 5. A list of active lecturers is displayed to the admin
Alternative Flows: [Alternative Flow 1 – Not in Network]	1. If the admin session is validated, they can directly input the link manually to reach their destination.
Exceptions:	1. The admin disconnected from the internet. 2. Admin was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal

	3. A user will have an active internet connection

3.2.2.2 UC-13: View Active Lecturers

Use Case ID:	UC-13
Use Case Name:	View Active Lecturers
Actors:	Admin
Description:	An Admin can view all the current active lecturers list on the portal who are currently teaching a programing related subject.
Trigger:	The admin clicks on the view lecturer's navigation link under Manage Lecturers.
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicked on the View Lecturers Nav Link
Postconditions:	POST-1. The admin can now view of all the active lecturers list. POST-2. The admin can now click on any one of the lecturers to see a summary of their performance history, edit their details, set them as inactive or delete them from the database.
Normal Flow:	1. The admin has validated and is logged in. 2. The admin is now viewing his dashboard. 3. The admin hovers over the drop down "Manage Lecturers." 4. Under the Manage Lecturers section the admin clicks on the link "View Active Lecturers" 5. A list of active lecturers is displayed to the admin
Alternative Flows: [Alternative Flow 1 – Not in Network]	1. If the admin session is validated, they can directly input the link manually to reach their destination.
Exceptions:	1. The admin disconnected from the internet. 2. Admin was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.3 UC-14: Review approval requests

Use Case ID:	UC-14
Use Case Name:	Review approval requests
Actors:	Admin
Description:	An admin can review approval requests. They will include update of a

	lecturer's account, student's account, challenge task. Deletion of a student's account or a challenge task, etc.
Trigger:	The admin clicks on review approval request in the navigation
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicked on the review approval request link
Postconditions:	POST-1. All the pending approvals are viewed by the admin.
Normal Flow:	1. The user clicks on the review approval request link. 2. All the pending requests are displayed by the system. 3. The admin can now approve or disapprove the requests.
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.4 UC-15: Change Lecturer status

Use Case ID:	UC-15
Use Case Name:	Change lecturer status
Actors:	Admin
Description:	Admin can change the lecturer's status as active or inactive
Trigger:	The admin clicks on the change status link Infront of the target lecturer
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicks on the view lecturers link under manage lecturer section. PRE-3. Admin clicks on the change status link Infront of the target user
Postconditions:	POST-1. The admin is now able to set the lecturer status as active or inactive.
Normal Flow:	1. The admin has validated and is logged in. 2. Admin clicks on the view lecturers link under manage lecturer section. 3. Admin clicks on the change status link Infront of the target user. 4. The admin is now able to set the lecturer status as active or inactive. 5. The system displays the message status changed
Alternative	None

Flows: [Alternative Flow 1 – Not in Network]	
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.5 UC-16: Edit Lecturer Details

Use Case ID:	UC-16
Use Case Name:	Edit Lecturer Details
Actors:	Admin, Lecturer
Description:	An Admin can edit a lecturer’s details. A Lecturer can also edit their details but only upon the admissions approval will they be changed.
Trigger:	The admin clicks on the edit lecturer navigation link Infront of the target lecturer.
Preconditions:	For Admin: PRE-1. Admin has validated and logged in. PRE-2. Admin clicked on the view active or inactive lecturers nav link. PRE-3. Admin clicks on the edit lecturer link Infront of the target lecturer. For Lecturer: PRE-1. A lecturer had validated and logged in. PRE-2. A lecturer clicks on profile link. PRE-3. A lecturer clicks on the update profile nav link
Postconditions:	For Admin: POST-1. The admin can now edit the lecturer’s details. POST-2. Upon changes the admin will click on the update button to confirm the changes. For Lecturer: POST-1. The Lecturer cannot edit his details. POST-2. The Lecturer can click on the update button to confirm the changes. POST-3. The changes will be submitted to the admin for review.
Normal Flow:	For Admin: 1. The admin has validated and is logged in. 2. The admin clicks on the view active or inactive lecturer nav link. 3. Admin clicks on the edit lecturer’s link Infront of the target lecturer.

	4. The admin updates the necessary details. 5. The admin clicks on the update button. 6. The screen displays a message "Updated." For Lecturer: 1. The lecturer has validated and logged in. 2. The lecturer clicks on the profile. 3. The lecturer clicks on the update profile link. 4. The lecturer edits the necessary fields. 5. The lecturer clicks on the update button. 6. The screen displays the message "The updates have been sent to the admin for review"
Alternative Flows: [Alternative Flow 1 – Not in Network]	1. If the admin and lecturer session is validated, they can directly input the link manually to reach their destination.
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.6 UC-17: Delete Lecturer

Use Case ID:	UC-17
Use Case Name:	Delete Lecturer
Actors:	Admin
Description:	Admin can delete the lecturer
Trigger:	The admin clicks on the delete button Infront of the target lecturer
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicks on the view lecturers link under manage lecturer section. PRE-3. Admin clicks on delete button Infront the target lecturer
Postconditions:	POST-1. The admin has now successfully deleted the lecturer.
Normal Flow:	1. The admin has validated and is logged in. 2. Admin clicks on the view lecturers link under manage lecturer section. 3. Admin clicks on the delete button. 4. The system asks for a confirmation if they really want to delete the

	lecturer. 5. The admin presses yes. 6. The lecturer is deleted successfully
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.7 UC-18: Upload Lecturer Excel

Use Case ID:	UC-18
Use Case Name:	Upload lecturers excel
Actors:	Admin
Description:	Admin can upload an excel file containing info of lecturers. They will be included into the system.
Trigger:	The admin clicks on the upload lecturer excel link under manage lecturers
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicks on the upload lecturer excel link under manage lecturer section. PRE-3. Admin selects an excel sheet to upload. PRE-4. Admin presses upload
Postconditions:	POST-1. The admin has successfully uploaded the excel sheet. POST-2. The uploaded lecturers can now be viewed under inactive lecturers list.
Normal Flow:	1. The admin has validated and is logged in. 2. Admin clicks on the upload lecturer excel link under manage lecturer section. 3. Admin selects an excel sheet to upload. 4. Admin presses upload 5. The system displays a InProgress message. 6. Upon successful upload the system generates a message ‘Uploaded’ 7. The uploaded lecturers can now be viewed under inactive lecturers list
Alternative	None

Flows: [Alternative Flow 1 – Not in Network]	
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.8 UC-19: Download Lecturer Excel

Use Case ID:	UC-19
Use Case Name:	Download lecturers excel
Actors:	Admin
Description:	Admin can download an excel file containing info of lecturers. That are included in the system
Trigger:	The admin clicks on the download lecturer excel link under manage lecturers
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicks on the download lecturer excel link under manage lecturer section
Postconditions:	POST-1. The admin has successfully downloaded the excel sheet.
Normal Flow:	1. The admin has validated and is logged in. 2. Admin clicks on the download lecturer excel link under manage lecturer section. 3. Admin presses download. 4. The system displays a InProgress message. 5. Upon successful import the system generates a message 'Imported'
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None

Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.9 UC-20: Assign Lecturer Section

Use Case ID:	UC-20
Use Case Name:	Assign Lecturer Section
Actors:	Admin
Description:	Admin can assign a section to a lecturer thus automatically changing their status to active
Trigger:	The admin clicks on the assign lecturer link in the section page.
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicks in the current sections link under Manage sections navigation. PRE-3. Admin selects a section he/she wants to assign a Lecturer. PRE-4. Section’s page has an assign a lecturer button which the admin will press to assign the lecturer. PRE-5. A list of all the active and inactive lecturers is displayed. PRE-6. Admin selects the lecturer he/she wants assign.
Postconditions:	POST-1. The lecturer is now assigned to the students, and he/she is now able to teach and assess the students.
Normal Flow:	1. Admin clicks in the current sections link under Manage sections navigation. 2. Admin selects a section he/she wants to assign a Lecturer. 3. Admin selects a section he/she wants to assign a Lecturer. 4. Section’s page has an assign a lecturer button which the admin will press to assign the lecturer. 5. A list of all the active and inactive lecturers is displayed. 6. Admin selects the lecturer he/she wants assign. 7. The system displays a message “Successfully assigned.” 8. The lecturer is now assigned to the students, and he/she is now able to teach and assess the students.
Alternative Flows: [Alternative Flow 1 – Not in Network]	4.1. The assign a lecturer button is not available because the max number of Lecturers that can be assigned has been reached. 4.2. The admin first unassigns a lecturer 4.3. We no return to step 4 of the normal flow
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None

Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.10 UC-21: Unassign Lecturer Section

Use Case ID:	UC-21
Use Case Name:	Unassign Lecturer Section
Actors:	Admin
Description:	Admin can Unassign a section assigned to a lecturer
Trigger:	The admin clicks on the Unassign lecturer link in the section page.
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicks in the current sections link under Manage sections navigation. PRE-3. Admin selects a section he/she wants to Unassign the Lecturer. PRE-4. Section's page has a Unassign a lecturer button which the admin will press to Unassign the lecturer. PRE-5. Admin selects the lecturer he/she wants Unassign.
Postconditions:	POST-1. The lecturer is now Unassign from the section.
Normal Flow:	1. Admin has validated and logged in. 2. Admin clicks in the current sections link under Manage sections navigation. 3. Admin selects a section he/she wants to Unassign the Lecturer. 4. Section's page has a Unassign a lecturer button which the admin will press to Unassign the lecturer. 5. Admin selects the lecturer he/she wants Unassign. 6. The system generates a message successfully unassigned.
Alternative Flows: [Alternative Flow 1 – Not in Network]	4.1. The Unassign a lecturer button is not available there are no lecturers assigned to the section. 4.2. The admin must first Assigns a lecturer. 4.3. Then we return to step 4 of the normal flow
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.11 UC-22: Add new section.

Use Case ID:	UC-22
Use Case Name:	Add new section
Actors:	Admin
Description:	Admin can add a new section to the portal
Trigger:	The admin clicks on the add new section link under the manage sections navigation
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. The admin has clicked on the add new section link under manage sections. PRE-3. The admin inputs the values of the section to be included. PRE-4. The admin clicks on add section button
Postconditions:	POST-1. The new section has been added now a lecturer can assign a lecturer to it and add students to it.
Normal Flow:	1. Admin has validated and logged in. 2. The admin has clicked on the add new section link under manage sections. 3. The admin inputs the values of the section to be included. 4. The admin clicks on add section button. 5. The system generates a message successfully added a section. 6. The new section has been added now a lecturer can assign a lecturer to it and add students to it.
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.12 UC-23: Remove section

Use Case ID:	UC-23
Use Case Name:	Remove section
Actors:	Admin
Description:	Admin can remove a section

Trigger:	The admin clicks on the remote section button Infront of the target section
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. The admin has clicked on the remote section button Infront of the target section
Postconditions:	POST-1. The admin has successfully removed a section.
Normal Flow:	1. Admin has validated and logged in. 2. The admin has clicked on the remote section link Infront of the target section. 3. The system generates a message are you sure you want to remove this section. 4. The admin clicks yes. 5. The section is removed
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device. 3. The admin clicks on no.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.13 UC-24: View section

Use Case ID:	UC-24
Use Case Name:	View sections
Actors:	Admin
Description:	Admin can view a section
Trigger:	The admin clicks on the view sections link under the manage sections navigation
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. The admin has clicked on the view sections link under manage sections
Postconditions:	POST-1. The admin can now view all the current sections active on the portal.
Normal Flow:	1. Admin has validated and logged in. 2. The admin has clicked on the view sections link under manage

	sections. 3. The system views all the sections currently on the system.
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.14 UC-25: Upload student excel.

Use Case ID:	UC-25
Use Case Name:	Upload students excel
Actors:	Admin
Description:	Admin can upload an excel file containing info of students. They will be included into the system.
Trigger:	The admin clicks on the upload students excel link under manage students
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicks on the upload students excel link under manage students' section. PRE-3. Admin selects an excel sheet to upload. PRE-4. Admin presses upload
Postconditions:	POST-1. The admin has successfully uploaded the excel sheet. POST-2. The uploaded students can now be viewed in the students list.
Normal Flow:	1. The admin has validated and is logged in. 2. Admin clicks on the upload students excel link under manage students' section. 3. Admin selects an excel sheet to upload. 4. Admin presses upload 5. The system displays a InProgress message. 6. Upon successful upload the system generates a message 'Uploaded' 7. The uploaded students can now be viewed under the students list
Alternative Flows: [Alternative Flow 1 – Not in	None

Network]	
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.2.15 UC-26: Download student excel.

Use Case ID:	UC-26
Use Case Name:	Download students excel
Actors:	Admin
Description:	Admin can download an excel file containing info of students. That are included in the system
Trigger:	The admin clicks on the download students excel link under manage students
Preconditions:	PRE-1. Admin has validated and logged in. PRE-2. Admin clicks on the download students excel link under manage lecturer section
Postconditions:	POST-1. The admin has successfully downloaded the excel sheet.
Normal Flow:	1. The admin has validated and is logged in. 2. Admin clicks on the download students excel link under manage lecturer section. 3. Admin presses download. 4. The system displays a InProgress message. 5. Upon successful import the system generates a message 'Imported'
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.3 Lecturer Perspective:

3.2.3.1 UC-27: Create lab task.

Use Case ID:	UC-27
Use Case Name:	Create lab task
Actors:	Lecturer
Description:	The lecturer can create a lab task
Trigger:	The lecturer clicks on create lab task link under Lab Tasks Navigation
Preconditions:	PRE-1. Lecturer clicks on the create lab task link. PRE-2. Lecturer inputs all the necessary values required to create a lab task. PRE-3. Lecturer submits the lab task.
Postconditions:	POST-1. The lab tasks are now available to be assigned.
Normal Flow:	1. The lecturer clicks on the create lab task link under manage Lab Tasks navigation 2. The lecturer inputs all the necessary values required to create a lab task. 3. The lecture submits the lab task. 4. The system displayed a message "Success." 5. The lab tasks are now available to assign to students
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.3.2 UC-28: View lab task

Use Case ID:	UC-28
Use Case Name:	View lab task
Actors:	Lecturer
Description:	The lecturer can view lab tasks
Trigger:	The lecturer clicks on view lab tasks link under Lab Tasks Navigation

Preconditions:	PRE-1. Lecturer clicks on the view lab tasks link.
Postconditions:	POST-1. The lab tasks are displayed by the system.
Normal Flow:	1. The lecturer clicks on the view lab tasks link under manage Lab Tasks navigation 2. The lab tasks re displayed to the lecturer
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.3.3 UC-29: Update lab task

Use Case ID:	UC-29
Use Case Name:	Update lab task
Actors:	Lecturer
Description:	The lecturer can update lab tasks
Trigger:	The lecturer clicks on update lab task button Infront of the target lab task
Preconditions:	PRE-1. Lecturer clicks on update lab task button Infront of the target lab task. PRE-2. The lecturer inputs the updated input. PRE-3. The lecturer presses submit.
Postconditions:	POST-1. The lab task is successfully updated.
Normal Flow:	1. The lecturer clicks on the view lab tasks link under Manage Lab tasks navigation. 2. Lecturer clicks on update lab task button Infront of the target lab task. 3. The lecturer inputs the updated input. 4. The lecturer presses submit.
Alternative Flows: [Alternative Flow	None

1 – Not in Network]	
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.3.4 UC-30: Assign lab task

Use Case ID:	UC-30
Use Case Name:	Assign Lab task
Actors:	Lecturer
Description:	The lecturer can assign a lab task to students. The lab task can be assigned by a teacher own custom lab tasks or a lab task from a pool of challenge tasks.
Trigger:	The lecturer clicks on the assign lab task button on the section summary page
Preconditions:	PRE-1. Lecturer has not already assigned a lab task. PRE-2. Lecturer presses assign a lab task button.
Postconditions:	POST-1. The lab task is successfully assigned to students.
Normal Flow:	1. The lecturer clicks on view sections under Manage sections navigation. 2. Lecturer selects the section he/she wants to assign a task to 3. The lecturer clicks the assign task button. 4. A list of tasks is displayed Infront of the lecturer. 5. The lecturer selects a task to assign. 6. The system generates a message “Task assigned”
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.3.5 UC-31: Unassign lab task

Use Case ID:	UC-31
Use Case Name:	Unassign Lab task
Actors:	Lecturer
Description:	The lecturer can unassign a lab task assigned to students.
Trigger:	The lecturer clicks on the unassign lab task button on the section summary page
Preconditions:	PRE-1. Lecturer has already assigned a lab task. PRE-2. Lecturer presses unassign a lab task button
Postconditions:	POST-1. The lab task is successfully unassigned.
Normal Flow:	7. The lecturer clicks on view sections under Manage sections navigation. 8. Lecturer selects the section he/she wants to unassign a task to 9. The lecturer clicks the unassign task button. 10. The system generates a message "Task unassigned"
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	3. The user disconnected from the internet. 4. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	4. A user will have a register account on the portal. 5. A user will a device through which they can login to the portal 6. A user will have an active internet connection

3.2.3.6 UC-32: Delete task

Use Case ID:	UC-32
Use Case Name:	Delete task
Actors:	Lecturer
Description:	The lecturer can delete a lab task he/she created themselves
Trigger:	The lecturer presses the delete button Infront of the target lab task
Preconditions:	PRE-1. Lecturer has validated and is logged in. PRE-2. Lecturer is currently viewing the lab tasks page. PRE-3. The lab task is not currently assigned. PRE-4. The lecturer has pressed the delete button Infront of the target lab task.

Postconditions:	POST-1. The lab task is successfully deleted.
Normal Flow:	1. The lecturer clicks on view lab tasks link under manage lab tasks page 2. The lecturer clicks on delete button Infront of the target lab task. 3. The system asks for confirmation. 4. The lecturer confirms. 5. The lab task is successfully deleted
Alternative Flows: [Alternative Flow 1 – Not in Network]	1.2.The lab task is already assigned. 1.3.System asks if the user would like to un assign it. 1.4.The lecturer confirms. 1.5.We return to the step 3 of the normal flow
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.3.7 UC-33: Review student requests

Use Case ID:	UC-33
Use Case Name:	Review student requests
Actors:	Lecturer
Description:	The lecturer can view all the students pending requests. It can be request for a lab task answer or to join the section.
Trigger:	The lecturer clicks on the view pending requests link on the navigation
Preconditions:	PRE-1. Lecturer has validated and is logged in. PRE-2. Lecturer clicked on the view pending requests link.
Postconditions:	POST-1. The lecturer is now viewing all the pending requests. POST-2. The lecturer can now accept or reject requests.
Normal Flow:	1. The lecturer logged in. 2. The lecturer clicked on the view pending requests
Alternative Flows: [Alternative Flow 1 – Not in Network]	None

Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.3.8 UC-34: Grade students

Use Case ID:	UC-34
Use Case Name:	Grade students
Actors:	Lecturer
Description:	The lecturer can grade a student based on their lab performance
Trigger:	The lecturer inputs marks and submits them after assessing the student performance.
Preconditions:	PRE-1. Lecturer has validated and is logged in. PRE-2. Lecturer is currently on the review student performance page. PRE-3. Lecturer viewed the student's lab task. PRE-4. Lecturer inputted some marks and pressed grade button.
Postconditions:	POST-1. The student has been successfully graded the student.
Normal Flow:	1. The lecturer clicks on view sections under Manage sections navigation. 2. Lecturer clicks on the lab attendees' link. 3. Lecturer clicks on the target student's name. 4. Lecturer is now viewing the student's performance in previous labs and can also assess the tasks of today's lab. 5. The lecturer clicks on the grade lab button Infront of the target lab. 6. The lecturer inputs marks 7. The lecturer clicks on grade button. 8. The system generates a message "Successfully graded"
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	2. The user disconnected from the internet. 3. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None

Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.3.9 UC-35: Review student performance

Use Case ID:	UC-35
Use Case Name:	Review Student performance
Actors:	Lecturer
Description:	The lecturer can review the students' performance in today's task or in previous tasks.
Trigger:	The lecturer clicks on the student's name under the lab attendees list.
Preconditions:	PRE-1. Lecturer has validated and is logged in. PRE-2. Lecturer is currently viewing the section summary page. PRE-3. Lecturer is clicked on the student's name under lab attendees' tab.
Postconditions:	POST-1. The system is displaying the current lab task student has attempted and can grade it accordingly.
Normal Flow:	1. The lecturer clicks on view sections under Manage sections navigation. 2. Lecturer clicks on the lab attendees' link. 3. Lecturer clicks on the target student's name. 4. Lecturer is now viewing the student's performance in previous labs and can also assess the tasks of today's lab.
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.4 Student Perspective:

3.2.4.1 UC-36: Attempt task

Use Case ID:	UC-36
Use Case Name:	Attempt task
Actors:	Student
Description:	The student can attempt a lab task
Trigger:	The student clicks on the attempt task button in the task detail page
Preconditions:	PRE-1. Student has validated and is logged in. PRE-2. Student is currently viewing the task page. PRE-3. Student pressed on the attempt task button
Postconditions:	POST-1. The student is now attempting a task on the IDE.
Normal Flow:	1. The student clicked on the view section summary link under manage sections. 2. The student clicked on assigned tasks link. 3. The student clicked on a task. 4. The student clicked on the attempt task button. 5. The system displays the IDE along with some general info and guidelines.
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.4.2 UC-37: Quit task

Use Case ID:	UC-37
Use Case Name:	Quit task
Actors:	Student
Description:	The student can quit a task
Trigger:	The student clicks on quit task button on the task attempt page.
Preconditions:	PRE-1. Student has validated and is logged in. PRE-2. Student is currently attempting the task. PRE-3. The student clicked on the quit task button.

Postconditions:	POST-1. The student has quitted the task and will be marked 0 if he does not re attempt it in time.
Normal Flow:	1. The student viewed the task. 2. The student clicked on attempt task. 3. The student clicked on quit task. 4. The system asks for confirmation. 5. The student confirms.
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.4.3 UC-38: Request solution

Use Case ID:	UC-38
Use Case Name:	Request Solution
Actors:	Student
Description:	The student can request the solution of a task
Trigger:	The student clicks on request solution button on the task attempt page.
Preconditions:	PRE-1. Student has validated and is logged in. PRE-2. Student is currently attempting the task. PRE-3. The student clicked on the request solution button.
Postconditions:	POST-1. The student request has been submitted to the lecturer.
Normal Flow:	6. The student viewed the task. 7. The student clicked on attempt task. 8. The student clicked on request solution button. 9. The system confirms with a message “Request sent to Lecturer”
Alternative Flows: [Alternative Flow 1 – Not in	None

Network]	
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.4.4 UC-39: View grades

Use Case ID:	UC-39
Use Case Name:	View grades
Actors:	Student
Description:	The student can view all his/her grades in lab tasks
Trigger:	The student clicks on grades link in the navigation
Preconditions:	PRE-1. Student has validated and is logged in. PRE-2. The student clicked on grades link in the navigation.
Postconditions:	POST-1. The student can view the grades he has got under a particular lecturer.
Normal Flow:	1. The student clicked on the grades link on the navigation. 2. The student can now view their lab grades under a specific lecturer.
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.4.5 UC-40: View tasks history

Use Case ID:	UC-40
Use Case Name:	View tasks history
Actors:	Student
Description:	The student can view all the tasks he/she has attempted until now

Trigger:	The student clicks on view task history link under Task's navigation
Preconditions:	PRE-1. Student has validated and is logged in. PRE-2. The student is currently viewing the PRE-2. Student clicked on the view task history link under Task navigation.
Postconditions:	POST-1. All the previously attempted tasks are viewed to the student
Normal Flow:	1. The student clicked on the section summary link. 2. The student clicked on the Tasks Tab. 3. The student clicked on the task's history link. 4. All the previously attempted tasks are being viewed by the student
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.5 Admin and Lecturer Perspective:

3.2.5.1 UC-41: Edit student's info.

Use Case ID:	UC-41
Use Case Name:	Edit Student's info
Actors:	Admin, Lecturer
Description:	An Admin and a Lecturer can edit a student's info
Trigger:	The user clicks on update student button after changing the student's values
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User clicks on the edit link Infront of the target student. PRE-3. The user has changed some values of the student. PRE-4. The user clicks on the update button.
Postconditions:	POST-1. The student's values are changed.
Normal Flow:	1. The user clicks on the view students link under manage student's navigation.

	2. User clicks on the edit link Infront of the target student. 3. The user has changed some values of the student. 4. The user clicks on the update button. 5. The system displays a message successfully updated.
Alternative Flows: [Alternative Flow 1 – Not in Network]	1.1. The user clicks on view sections link under manage sections navigation. 1.2. The user selects the section which may have the required student. 1.3. We return to the step 2 of normal flow
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.5.2 UC-42: View section students

Use Case ID:	UC-42
Use Case Name:	View Section students
Actors:	Admin, Lecturer
Description:	An Admin and a Lecturer can view students under a specific section
Trigger:	The user clicks on the section link on the view sections page
Preconditions:	PRE-1. User has validated and logged in. PRE-2. The user clicks on the view sections link under Manage sections navigation. PRE-3. The user clicks on the target section to display all its details
Postconditions:	POST-1. The students registered under the section are visible on the page.
Normal Flow:	1. User has validated and logged in. 2. The user clicks on the view sections link under Manage sections navigation. 3. The user clicks on the target section to display all its details. 4. The students registered under the section are visible on the page
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their

	account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.5.3 UC-43: Delete student

Use Case ID:	UC-43
Use Case Name:	Delete Student
Actors:	Admin, Lecturer
Description:	An Admin and a Lecturer can delete a student
Trigger:	The user clicks on delete button Infront of the target user
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User clicks on the delete button Infront of the target user.
Postconditions:	POST-1. The student is deleted, and his/her account is reset.
Normal Flow:	1. The user clicks on the view students link under manage student's navigation. 2. User clicks on the delete button Infront of the target student. 3. The system asks for confirmation. 4. The user presses yes 5. the student's account is deleted.
Alternative Flows: [Alternative Flow 1 – Not in Network]	1.1. The user clicks on view sections link under manage sections 1.2. navigation 1.3. The user selects the section which may have the required student. We return to the step 2 of normal flow
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.5.4 UC-44: View challenge task

Use Case ID:	UC-44
Use Case Name:	View challenge task

Actors:	Admin, Lecturer
Description:	An Admin and a Lecturer can view a challenge task.
Trigger:	The user clicks on view challenge task link under Manage challenge tasks navigation
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User clicks on view challenge task link under Manage challenge tasks navigation
Postconditions:	POST-1. The challenge tasks are now visible to the user.
Normal Flow:	1. The user clicks on view challenge task link under Manage challenge tasks navigation. 2. The challenge tasks are now being displayed to the user
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.5.5 UC-45: Delete student logs.

Use Case ID:	UC-45
Use Case Name:	Delete Student Logs
Actors:	Admin, Lecturer
Description:	An Admin and a Lecturer delete a student's logs
Trigger:	The user clicks on delete logs button Infront of the target user
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User clicks on the delete logs button Infront of the target user.
Postconditions:	POST-1. The student's loges are deleted, and his/her account is reset.
Normal Flow:	6. The user clicks on the view students link under manage student's navigation. 7. User clicks on the delete logs button Infront of the target student. 8. The system asks for confirmation. 9. The user presses yes 10. The student's account is reset.

Alternative Flows: [Alternative Flow 1 – Not in Network]	1.1. The user clicks on view sections link under manage sections navigation. 1.2. The user selects the section which may have the required student. 1.3. We return to the step 2 of normal flow
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.5.6 UC-46: Edit challenge task.

Use Case ID:	UC-46
Use Case Name:	Edit challenge task
Actors:	Admin, Lecturer
Description:	An Admin and a Lecturer can edit a challenge task.
Trigger:	The user clicks on edit challenge task button Infront of the target challenge task
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User clicked on edit button Infront the target challenge tasks. PRE-3. User has input some changes. PRE-4. User Pressed edit task button
Postconditions:	POST-1. The challenge task values have been successfully updated.
Normal Flow:	1. The user clicks on view challenge task link under Manage challenge tasks navigation. 2. The user clicks on the edit button Infront of the target challenge task. 3. The user has made some changes to the input. 4. The user pressed update task 5. The system displays a message "Successfully updated"
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None

Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.5.7 UC-47: Add challenge task.

Use Case ID:	UC-47
Use Case Name:	Add challenge task
Actors:	Admin, Lecturer
Description:	An Admin and a Lecturer can add a challenge task. When a lecturer adds a task, it is sent to the admin for approval.
Trigger:	The user clicks on add a challenge task button under the add challenge task page after inputting all the necessary info.
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User has input all necessary fields for the challenge task. PRE-3. The user has clicked on add task button
Postconditions:	POST-1. The challenge task is now available to assign to students.
Normal Flow:	1. The user clicks on add a challenge task link under the Manage challenge tasks navigation. 2. The user inputs all the necessary fields in the add challenge task page. 3. The user clicks on add the task button. 4. The system displays a message "The task has been added to the pool."
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.5.8 UC-48: Delete challenge task.

Use Case ID:	UC-48
Use Case Name:	Delete challenge task
Actors:	Admin, Lecturer
Description:	An Admin and a Lecturer can edit a challenge task.
Trigger:	The user clicks on edit challenge task button Infront of the target challenge task

Preconditions:	PRE-1. User has validated and logged in. PRE-2. User clicked on delete button Infront the target challenge tasks
Postconditions:	POST-1. The challenge task has been deleted.
Normal Flow:	1. The user clicks on view challenge task link under Manage challenge tasks navigation. 2. The user clicks on the delete button Infront of the target challenge task. 3. The system displays a message “Successfully deleted”
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.5.9 UC-49: View at risk students

Use Case ID:	UC-49
Use Case Name:	View at risk students
Actors:	Admin, Lecturer
Description:	A user can view at risk students. Those students who are having difficulties in learning.
Trigger:	The User clicks on at risk students link under manage student’s navigation
Preconditions:	PRE-1. User has validated and logged in. PRE-2. User clicked on view at risk students’ link.
Postconditions:	POST-1. All the at-risk students are displayed.
Normal Flow:	1. The user clicks on view at risk students link under manage student’s navigation. 2. The at-risk students are displayed by the system
Alternative Flows: [Alternative Flow 1 – Not in Network]	None

Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None
Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.2.6 Lecturer and Student Perspective:

3.2.6.1 UC-50: View at risk students

Use Case ID:	UC-50
Use Case Name:	View assigned lab task
Actors:	Lecturer, Student
Description:	The user can view assigned lab tasks under Task's navigation on the section summary page
Trigger:	The user clicks on view assigned lab tasks link under the section summary page
Preconditions:	PRE-1. User has validated and is logged in. PRE-2. User is currently viewing the section summary page. PRE-3. User has clicked on the view assigned lab tasks link
Postconditions:	POST-1. The student is now viewing the assigned lab tasks. POST-2. The user can click on any one of the assigned lab tasks to view its complete details.
Normal Flow:	1. The user clicks on the section summary page link under Manage sections navigation. 2. The user clicks on assigned labs link on the section summary page. 3. The user is viewing all the tasks. 4. The user clicks on a task. 5. The task details are displayed to the user
Alternative Flows: [Alternative Flow 1 – Not in Network]	None
Exceptions:	1. The user disconnected from the internet. 2. User was interrupted because somebody else logged into their account from a different device.
Business Rules	None

Assumptions:	1. A user will have a register account on the portal. 2. A user will a device through which they can login to the portal 3. A user will have an active internet connection

3.3 Functional Requirements

3.3.1 FR's Coinciding with all dashboard modules.

3.3.1.1 FR-1: Login

Identifier	FR-1
Title	Login
Requirement	The user will be able to login into the system
Source	User
Rationale	To keep unwanted users out of the system
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.1.2 FR-2: Logout

Identifier	FR-2
Title	Logout
Requirement	The user will be able to logout of the system
Source	User
Rationale	To allow user to terminate their session when they are done.
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.1.3 FR-3: User verification

Identifier	FR-3
Title	User verification
Requirement	The user will be verified and only then will be allowed to use the system
Source	User

Rationale	To keep unwanted users out of the system
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.1.4 FR-4: Login verification

Identifier	FR-4
Title	Login verification
Requirement	The system will verify user input and credentials
Source	User
Rationale	To provide a secure environment where invalid users do not login to a valid an account
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.1.5 FR-5: Session limit

Identifier	FR-5
Title	Session limit
Requirement	The system will limit one session per user
Source	User
Rationale	To ease the load on the server.
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.1.6 FR-6: Session termination

Identifier	FR-6
Title	Session termination
Requirement	The system will delete a user session when a new one is created.
Source	User
Rationale	To ease the load on the server

Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.1.7 FR-7: Check active session.

Identifier	FR-7
Title	Check active session
Requirement	The system will check for active sessions of a user
Source	User
Rationale	To terminate extra sessions
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.1.8 FR-8: See notifications

Identifier	FR-8
Title	Sees notifications
Requirement	The user will be able to see their notification
Source	User
Rationale	To keep a user updated of latest events
Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.1.9 FR-9: Delete notifications

Identifier	FR-9
Title	Delete notifications
Requirement	The user will be able to delete unwanted notifications
Source	User
Rationale	For managerial preferences of a user
Business Rule (if required)	None

Dependencies	None
Priority	Low

3.3.1.10 FR-10: View student rankings

Identifier	FR-10
Title	View student rankings
Requirement	The user will be able to view student rankings with respect to their sections
Source	User
Rationale	To summarize and show a student's performance in their respective sections
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.1.11 FR-11: View profile

Identifier	FR-11
Title	View profile
Requirement	The user will be able to view their profile
Source	User
Rationale	So, a user can review their own details and in the case of an Admin and a Lecturer even update them.
Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.1.12 FR-12: Update email

Identifier	FR-12
Title	Update email
Requirement	The user will be able to update their email address
Source	User
Rationale	So that a user can receive the latest mail on their current email address.

Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.1.13 FR-13: Update profile picture

Identifier	FR-13
Title	Update profile picture
Requirement	The user will be able to update profile picture
Source	User
Rationale	For managerial purposes
Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.1.14 FR-13: Recover password

Identifier	FR-13
Title	Recover password
Requirement	The user will be able to recover their password if they ever forget it
Source	User
Rationale	So that a user can login to their account with the new password if they forget the old one
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2 FR's coinciding with admin dashboard.

3.3.2.1 FR-14: View active lecturers.

Identifier	FR-14
Title	View active lecturers
Requirement	The admin will be able to view all active lecturers

Source	User
Rationale	So, the admin could know which lecturers are currently teaching students in a section.
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.2.2 FR-15: Change lecturer status to inactive

Identifier	FR-15
Title	Change lecturer status to inactive
Requirement	The admin will be able to change the status of a lecturer as inactive
Source	User
Rationale	If a lecturer is not able to attend classes anymore and admin can unassign him/her from all sections.
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.2.3 FR-16: Assign lecturer to a section

Identifier	FR-16
Title	Assign lecturer to a section
Requirement	The admin will be able to assign a lecturer to a section
Source	User
Rationale	So, a lecturer could conduct labs as per course requirement
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.2.4 FR-17: Unassign lecturer section

Identifier	FR-17
Title	Unassign a lecturer from section
Requirement	The admin will be able to unassign a lecturer from a section
Source	User
Rationale	If a lecturer no longer takes classes of a certain section, he/she will be removed from that section
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.2.5 FR-18: Check max assigned lecturers.

Identifier	FR-18
Title	Check max assigned lecturers
Requirement	The system will check the that not more than 2 lecturers are assigned to a section
Source	User
Rationale	So that no more than 2 lecturers are assigned to the same section for the same subject.
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2.6 FR-19: View inactive lecturers.

Identifier	FR-19
Title	View inactive lecturers
Requirement	The admin can view which lecturers are currently not assigned to any section
Source	User
Rationale	So that the admin can assign an inactive lecturer to a class as per

	instructions from the department
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2.7 FR-20: Edit lecturer details.

Identifier	FR-20
Title	Edit lecturer details
Requirement	The admin can edit a lecturer's details
Source	User
Rationale	In case of any misinput it can be corrected later
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2.8 FR-21: Add section

Identifier	FR-21
Title	Add section
Requirement	The admin can add a new section to the system
Source	User
Rationale	So, students can be assigned to that section
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.2.9 FR-22: Remove section

Identifier	FR-22

Title	Remove Section
Requirement	The admin can remove a section from the system
Source	User
Rationale	If a section has graduated and is no longer required in the system, it can be removed
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2.10 FR-23: Update section

Identifier	FR-23
Title	Update section
Requirement	The admin can update a section's details
Source	User
Rationale	In case off any miss input an admin can correct it later.
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2.11 FR-24: View all sections.

Identifier	FR-24
Title	View all sections
Requirement	An admin can view all current sections
Source	User
Rationale	SO, an admin can get an overview of the current sections in the database
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2.12 FR-25: View section lecturer

Identifier	FR-25
Title	Check max assigned lecturers
Requirement	The system will check the that not more than 2 lecturers are assigned to a section
Source	User
Rationale	So that no more than 2 lecturers are assigned to the same section for the same subject.
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2.13 FR-26: Deny lecturer assignment.

Identifier	FR-26
Title	De
Requirement	The system will check the that not more than 2 lecturers are assigned to a section
Source	User
Rationale	So that no more than 2 lecturers are assigned to the same section for the same subject.
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2.14 FR-27: Check duplicate lecturers.

Identifier	FR-27
Title	Check duplicate lecturers

Requirement	The system will check that no lecturer is inserted twice by the system
Source	User
Rationale	So that the system is not populated by duplicate data
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2.15 FR-28: Upload lecturers excel.

Identifier	FR-28
Title	Upload lecturers excel
Requirement	The admin will be able to upload lecturers excel sheet containing all their details
Source	User
Rationale	Uploading a lecturer one by one is a lengthy process. An admin can just upload them altogether through an excel sheet and can update them later
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.2.16 FR-29: Download lecturers excel.

Identifier	FR-29
Title	Download lecturers excel
Requirement	The admin can download the lecturers excel sheet
Source	User
Rationale	For backup purposes
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.2.17 FR-30: Upload student excel.

Identifier	FR-30
Title	Upload students excel
Requirement	The admin can upload student excel sheet
Source	User
Rationale	Uploading a student one by one is a lengthy process. An admin can just upload them altogether through an excel sheet and can update them later
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.2.18 FR-31: Download student excel.

Identifier	FR-31
Title	Download students excel
Requirement	The admin can download the students excel sheet
Source	User
Rationale	For backup purposes
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.2.19 FR-32: View approval requests

Identifier	FR-32
Title	View approval requests
Requirement	The admin can view approval requests of various purposes.
Source	User
Rationale	The admin can review approval request for various things such as a lecturer's request to update a challenge task

Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.2.20 FR-33: Approve/disapprove request.

Identifier	FR-33
Title	Approve/disapprove request
Requirement	The admin can approve or disapprove any request
Source	User
Rationale	An admin can accept or decline a request after reviewing it
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.3 FR's coinciding with lecturer dashboard.

3.3.3.1 FR-34: Create lab task.

Identifier	FR-34
Title	Create lab task
Requirement	The lecturer can create a lab task
Source	User
Rationale	A lecturer can create a lab task to assign to students
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.3.2 FR-35: View lab task

Identifier	FR-35

Title	View lab task
Requirement	The lecturer can view a lab task he/she created
Source	User
Rationale	The lecturer can view a lab task he//she created to review its contents
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.3.3 FR-36: Update lab task

Identifier	FR-36
Title	Update lab task
Requirement	The lecturer can update a lab task he/she created
Source	User
Rationale	The lecturer can update a lab task he/she created to edit any mistake
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.3.4 FR-37: Delete lab task.

Identifier	FR-37
Title	Delete lab task
Requirement	The lecturer can delete a lab task
Source	User
Rationale	The lecturer can delete unwanted lab tasks
Business Rule (if required)	None
Dependencies	None

Priority	High

3.3.3.5 FR-38: Assign lab task

Identifier	FR-38
Title	Assign lab task
Requirement	The lecturer can assign a lab task to students
Source	User
Rationale	The lab tasks can be assigned by lecturers to students
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.3.6 FR-39: Unassign lab task

Identifier	FR-39
Title	Unassign lab task
Requirement	The lecturer can unassign a lab task
Source	User
Rationale	The lecturer can unassign a lab task if it was assigned incorrectly
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.3.7 FR-40: View student requests

Identifier	FR-35
Title	View lab task
Requirement	The lecturer can view a lab task he/she created
Source	User

Rationale	The lecturer can view a lab task he//she created to review its contents
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.3.8 FR-41: Approve/Disapprove student requests.

Identifier	FR-41
Title	Approve/Disapprove student requests
Requirement	The lecturer can approve/disapprove student requests
Source	User
Rationale	Student's may request lab solutions from the lecturer. It is a lecturer's job to approve or disapprove that request
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.3.9 FR-42: Grade students

Identifier	FR-42
Title	Grade students
Requirement	The lecturer can grade students
Source	User
Rationale	The lecturer will be able to grade students upon their performance in lab
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.3.10 FR-43: Review student performance

Identifier	FR-43
Title	Review student performance
Requirement	The lecturer can review a student's performance in lab
Source	User
Rationale	A student will be graded every lab as per their performance
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.3.11 FR-44: Submit revisions to student task.

Identifier	FR-44
Title	Submit revisions to student task
Requirement	The lecturer can provide remarks on a student's lab performance
Source	User
Rationale	So that a student may know about his/her performance in lab
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.3.12 FR-45: Review challenge set

Identifier	FR-45
Title	Review challenge set
Requirement	The lecturer can review a challenge set
Source	User
Rationale	If a lecturer finds something wrong in a challenge task, he/she can submit it for revisions
Business Rule (if required)	None
Dependencies	None

Priority	Medium

3.3.4 FR's coinciding with student dashboard.

3.3.4.1 FR-46: Attempt lab task

Identifier	FR-46
Title	Attempt lab task
Requirement	The student can attempt a lab task
Source	User
Rationale	So, the student can attempt the task and be assessed by the lecturer.
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.4.2 FR-47: Quit lab task.

Identifier	FR-47
Title	Quit lab task
Requirement	The student can quit a lab task
Source	User
Rationale	If a student is unable to attempt the given task currently, he/she can always quit it to do it later within the time limit.
Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.4.3 FR-48: Request lab solution

Identifier	FR-48
Title	Request lab solution

Requirement	The student can request the solution of a lab task from the lecturer
Source	User
Rationale	If a student does not understand a lab task, they can always request the solution from the lecturer
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.4.4 FR-49: View personal grade.

Identifier	FR-49
Title	View personal grade
Requirement	The student can view their grades
Source	User
Rationale	So, a student can know the result of their assessment
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.4.5 FR-50: View attempted tasks.

Identifier	FR-50
Title	View attempted tasks
Requirement	The student can view previously attempted tasks
Source	User
Rationale	So, student can view where he/she made mistakes and learn from them later.
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.4.6 FR-51: Reattempt task for practice

Identifier	FR-51
Title	Reattempt task for practice
Requirement	The student can reattempt a task for practice
Source	User
Rationale	So, a student can prepare for their exams or self-study.
Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.5 FR's coinciding with admin and lecturer dashboards.

3.3.5.1 FR-52: Edit student info.

Identifier	FR-52
Title	Edit student info
Requirement	The admin and lecturer can edit a student's info
Source	User
Rationale	If any incorrect information was added it can be updated
Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.5.2 FR-53: Delete student

Identifier	FR-53
Title	Delete student
Requirement	The admin and lecturer can delete a student
Source	User
Rationale	If a student, no longer exists in the university he/she can be removed

Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.5.3 FR-54: View challenge task

Identifier	FR-54
Title	View challenge task
Requirement	The admin and lecturer can view challenge tasks
Source	User
Rationale	So, they can view the details of challenge task and their difficulty levels
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.5.4 FR-55: Delete challenge task.

Identifier	FR-55
Title	Delete a challenge task
Requirement	The admin and lecturer can delete a challenge task
Source	User
Rationale	Unnecessary challenge tasks can be deleted to clear up space in the database.
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.5.5 FR-56: Edit challenge task.

Identifier	FR-56
Title	Edit challenge task
Requirement	The admin and lecturer can edit a challenge task
Source	User
Rationale	If a challenge task requires some revisions, they can be done easily
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.5.6 FR-57: Test challenge task

Identifier	FR-57
Title	Test challenge task
Requirement	The admin and lecturer can test a challenge task
Source	User
Rationale	They can test to see if the challenge task meets expectations or not
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.5.7 FR-58: Verify challenge task input.

Identifier	FR-58
Title	Verify challenge task input
Requirement	The system will verify if the inputted expected results match the type of the expected result
Source	User
Rationale	If the challenge task is expected to provide a result in string value than the expected result value should be of a type string for comparison.
Business Rule (if required)	None

Dependencies	None
Priority	High

3.3.5.8 FR-59: View at risk students

Identifier	FR-59
Title	View at risk students
Requirement	The admin and lecturer can view at risk students
Source	User
Rationale	They can view at risk students so that lecturers can take necessary steps to help those students in need
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.5.9 FR-60: View section students

Identifier	FR-60
Title	View section students
Requirement	The admin and lecturer can view section students
Source	User
Rationale	They can view which students are registered under the specific section
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.5.10 FR-61: Delete student logs.

Identifier	FR-61
Title	Delete student logs
Requirement	The admin and lecturer can delete student logs
Source	User
Rationale	If a student requires their account to be reset due to some valid reasons or unnecessary logs were saved so they can be deleted.
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.6 FR's coinciding with student and lecturer dashboards.

3.3.7 FR's coinciding with rating and feedback.

3.3.7.1 FR-69: Read feedbacks

Identifier	FR-69
Title	Read feedbacks
Requirement	The user will be able to read feedbacks
Source	User
Rationale	To provide a transparent feedback platform
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.7.2 FR-70: Give feedback

Identifier	FR-70
Title	Give feedback
Requirement	The user will be able to give feedback
Source	User
Rationale	To provide a transparent feedback platform
Business Rule (if required)	None
Dependencies	None

Priority	Medium

3.3.7.3 FR-71: Delete feedback

Identifier	FR-71
Title	Delete feedback
Requirement	The user will be able to delete feedbacks
Source	User
Rationale	If a user wants to remove their given feedback for some reason, they can do so
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.7.4 FR-72: Review lecturer performance

Identifier	FR-72
Title	Review lecturer performance
Requirement	The student will be able to review their lecturer by the end of the semester
Source	User
Rationale	So, an admin can also assess what the students thought about a lecturer’s teaching methods
Business Rule (if required)	None
Dependencies	None
Priority	Medium

3.3.8 FR’s coinciding with IDE.

3.3.8.1 FR-73: Code compilation

Identifier	FR-73
Title	Code compilation
Requirement	The IDE will compile a given code
Source	User
Rationale	So, programming actives can be performed on the platform
Business Rule (if	None

required)	
Dependencies	None
Priority	High

3.3.8.2 FR-74: Result generation

Identifier	FR-74
Title	Result generation
Requirement	The IDE will generate code result after compilation
Source	User
Rationale	So, a user can know the output of their program
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.8.3 FR-75: Error handling

Identifier	FR-75
Title	Error handling
Requirement	The IDE will handle errors so that the website does not crash upon an incorrect code
Source	User
Rationale	Sometimes infinite loops or similar errors can cause a website to crash
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.8.4 FR-76: Error detailing

Identifier	FR-76
Title	Error detailing
Requirement	The IDE will generate errors and display details to users
Source	User
Rationale	So, user would get an insight on where he/she made a mistake in the program

Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.8.5 FR-77: Input handler

Identifier	FR-77
Title	Input handler
Requirement	The IDE will provide an input box for coding. It will be user friendly for ease of use
Source	User
Rationale	So, a user can program easily on the platform
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.8.6 FR-78: Task helping material.

Identifier	FR-78
Title	Task helping material
Requirement	The IDE will display helping material given by the lecturer.
Source	User
Rationale	So, user can code and if he /she gets stuck they can refer to the helping material.
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.9 FR's coinciding with Assessment.

3.3.9.1 FR-79: Compare user result and expected result.

Identifier	FR-79
Title	Compare user result and expected result

Requirement	The system will compare user result to expected result. If they match the user seems to have passed the task
Source	User
Rationale	For automatic assessment without lecturer intervention
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.9.2 FR-80: Check copied code.

Identifier	FR-80
Title	Check copied code
Requirement	The system will alert the lecturer if a student copy and pastes code from other resources.
Source	User
Rationale	For check and balance so that a student does not take the easy way out
Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.9.3 FR-81: Provide helping material from database.

Identifier	FR-81
Title	Provide helping material from database
Requirement	The system will provide relative helping materials is a student fails a task repetitively
Source	User
Rationale	So, a user can gain extra information if he/she is new to programming, or their concepts are not clear yet.
Business Rule (if required)	None
Dependencies	None
Priority	Loa

3.3.9.4 FR-82: Provide relative tasks.

Identifier	FR-82
Title	Provide relative tasks
Requirement	The system will provide relative easier tasks for practice if a user keeps on failing the current task
Source	User
Rationale	So, the student can gain more practice and understanding of programming.
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.9.5 FR-83: Set student as at risk.

Identifier	FR-83
Title	Set student as at risk
Requirement	The system will set a student as at risk if he/she is not meeting a certain criterion
Source	User
Rationale	So, the admin and lecturer can know which students are having a hard time adjusting to programming
Business Rule (if required)	None
Dependencies	None
Priority	High

3.3.9.6 FR-84: Unset student as at risk

Identifier	FR-84
Title	Unset student as at risk
Requirement	The system will unset a student as at risk if they meet a criterion
Source	User
Rationale	So, admin and lecturers can know about the real at-risk students
Business Rule (if required)	None
Dependencies	None

Priority	High

3.3.9.7 FR-85: Warn lecturer of suspicious activity.

Identifier	FR-85
Title	Warn lecturer of suspicious activity
Requirement	The system will warn the lecturer about suspicious activity such as if a student as not typed in each amount of time the system will deem them unresponsive and report them to the lecturer
Source	User
Rationale	For check and balance
Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.10 FR's coinciding with Application settings.

3.3.10.1 FR-86: Set theme

Identifier	FR-86
Title	Set theme
Requirement	A user can set their dashboard theme
Source	User
Rationale	For user friendliness
Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.10.2 FR-87: Set default theme.

Identifier	FR-87
Title	Set default theme
Requirement	The system will set default theme if no theme is set by the user
Source	User
Rationale	For user friendliness

Business Rule (if required)	None
Dependencies	None
Priority	Low

3.3.10.3 FR-88: Mute notifications

Identifier	FR-88
Title	Mute notifications
Requirement	A user can mute their notifications
Source	User
Rationale	For user friendliness
Business Rule (if required)	None
Dependencies	None
Priority	Low

3.4 Non-Functional Requirements

3.4.1 Usability

USE-1	A new user shall easily learn the application within 3 to 5 times of using the application.
USE-2	A trained user shall be able to perform tasks in less than 3 minutes.
USE-3	90 percent of users shall be able to perform major tasks without requiring any help.

3.4.2 Performance

PER-1	The searches performed within the system in any module will take no more than 4 seconds to process.
PER-2	Any request in the system will be completed within 4 seconds.
PER-3	The verification code will be sent within 3 seconds after the user has submitted the form in account creation page.

3.4.3 Scalability

SCB-1	The initial system shall be able to support up to a maximum of 200 users.
SCB-2	The initial system shall be able to handle up to 40 concurrent users at a time by hosting this application on a cloud hosting platform.

3.4.4 Security

SEC-1	The system shall perform hashing of password to protect against unauthorized access.

3.4.5 Reliability

REL-1	The system shall be available at least 95 percent of the time by hosting this application on cloud hosting platform.
REL-2	The average time to repair any fault should not be more than 3 hours.

4 Design and Architecture

The software design methodology and software process model we will use for the development of Student Assessment Platform application is as follows.

4.1 System Architecture

System will consist of many major components. These will be implemented as classes and designed as functional unit. Student, Admin and Lecturer work as a core component. Therefore, all components depend on them. Assignment, labs, and tasks component will depend on the lecturer as they will upload it for the student.

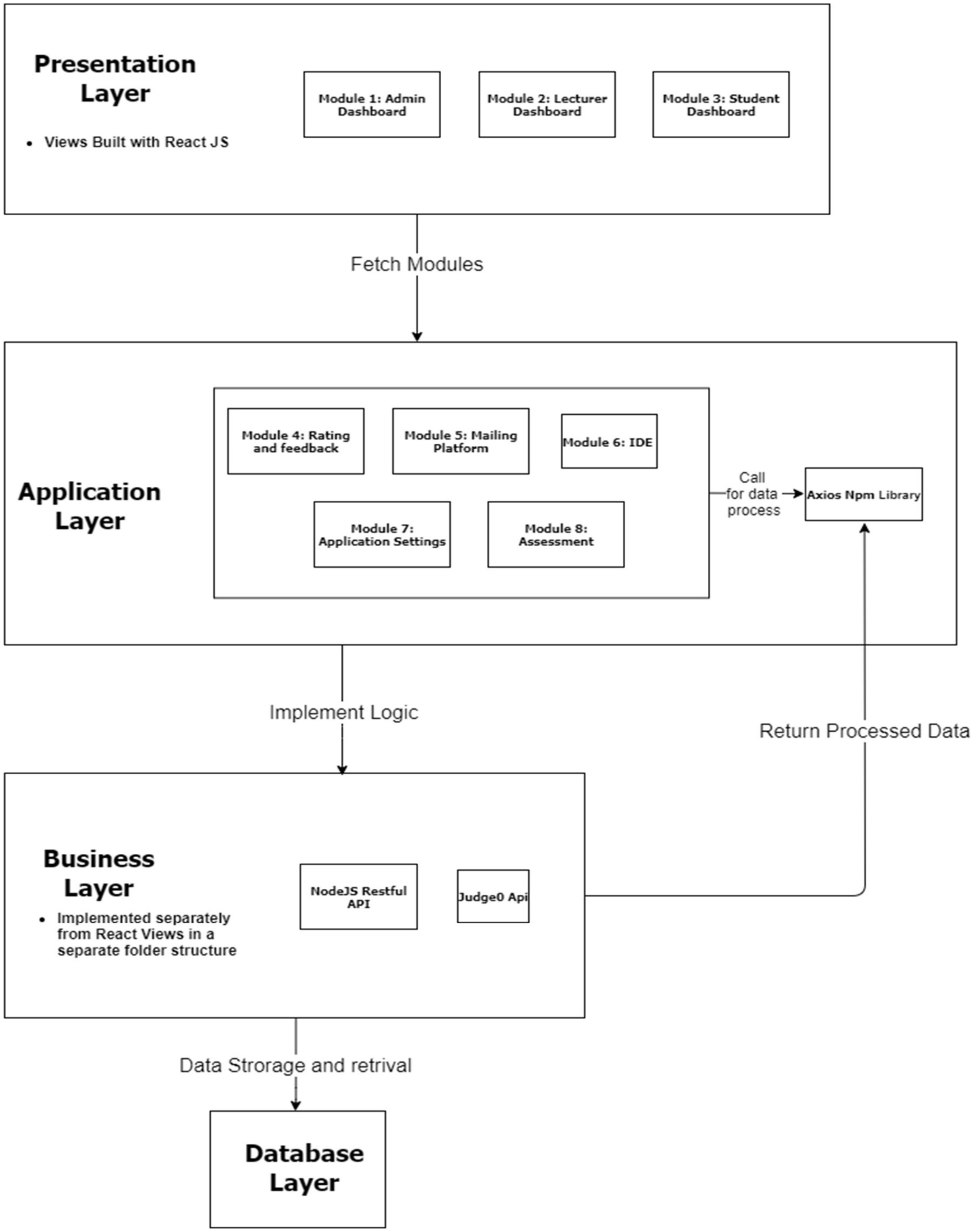

Figure 3 System Architecture Diagram

4.2 Data Representation

"Student Assessment Platform" has a SQL Database. It uses MySQL for storge of data.

4.2.1 Data dictionary

```
`admin` (
  `adminId` int (11) NOT NULL,
  `email` varchar (100) NOT NULL,
  `userName` varchar (100) NOT NULL,
  `password` varchar (100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.


`assignedwork` (
  `workId` int (11) NOT NULL,
  `assigmentId` int (11) NOT NULL,
  `sectionId` int (11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.

 `assignment` (
  `assignmentID` int (11) NOT NULL,
  `assignmentDetails` varchar (1000) NOT NULL,
  `timeAssigned` date DEFAULT current_timestamp (),
  `deadline` date NOT NULL,
  `helpingMaterial` varchar (100) NOT NULL,
  `solution` varchar (100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.

 `lecturer` (
  `lecturerId` int (11) NOT NULL,
  `userName` varchar (100) NOT NULL,
  `password` varchar (100) NOT NULL,
  `email` varchar (100) NOT NULL,
  `name` varchar (100) NOT NULL,
  `status` tinyint (1) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.

 `lecturerassigned` (
  `assignId` int (11) NOT NULL,
  `lecturerId` int (11) NOT NULL,
  `subjectId` int (11) NOT NULL,
  `sectionId` int (11) NOT NULL,
  `programmingLanguageId` int (11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.
```

```
 `mail` (
  `mailId` int (11) NOT NULL,
  `sender` varchar (100) NOT NULL,
  `receiver` varchar (100) NOT NULL,
  `message` varchar (100) NOT NULL,
  `attachmentsLink` varchar (100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.

 `notification` (
  `notiId` int (11) NOT NULL,
  `notification` varchar (100) NOT NULL,
  `relativeLink` varchar (100) NOT NULL,
  `userId` int (11) NOT NULL,
  `userType` varchar (100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.

 `programminglanguage` (
  `programmingLanguageId` int (11) NOT NULL,
  `programmingLanguage` varchar (100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.


 `section` (
  `sectionId` int (11) NOT NULL,
  `section` varchar (100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.



 `student` (
  `studentId` int (11) NOT NULL,
  `userName` varchar (100) NOT NULL,
  `password` varchar (100) NOT NULL,
  `name` varchar (100) NOT NULL,
  `rollNo` varchar (100) NOT NULL,
  `email` varchar (100) NOT NULL,
  `sectionId` int (11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.

 `subject` (
  `subjectId` int (11) NOT NULL,
  `subject` varchar (100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=latin1.
```

4.3 Process Flow/Representation

System architecture is the conceptual model that defines the behavior, structure, and views of a system. An architecture description is a formal description and representation of a system, organized in a way that supports reasoning about the behaviors and structures of the system. **Student Assessment Platform** will follow hybrid architecture concept. To meet design and development standards we will be using MVC architecture at application level. Client side will have MVC (Model-View-Controller) pattern which will communicate with server. In general, it makes up as client, logic/business layer, database server as N-Tier (3-Tier) Architecture as the business layer is handled by node/express and database layer as mongo server.

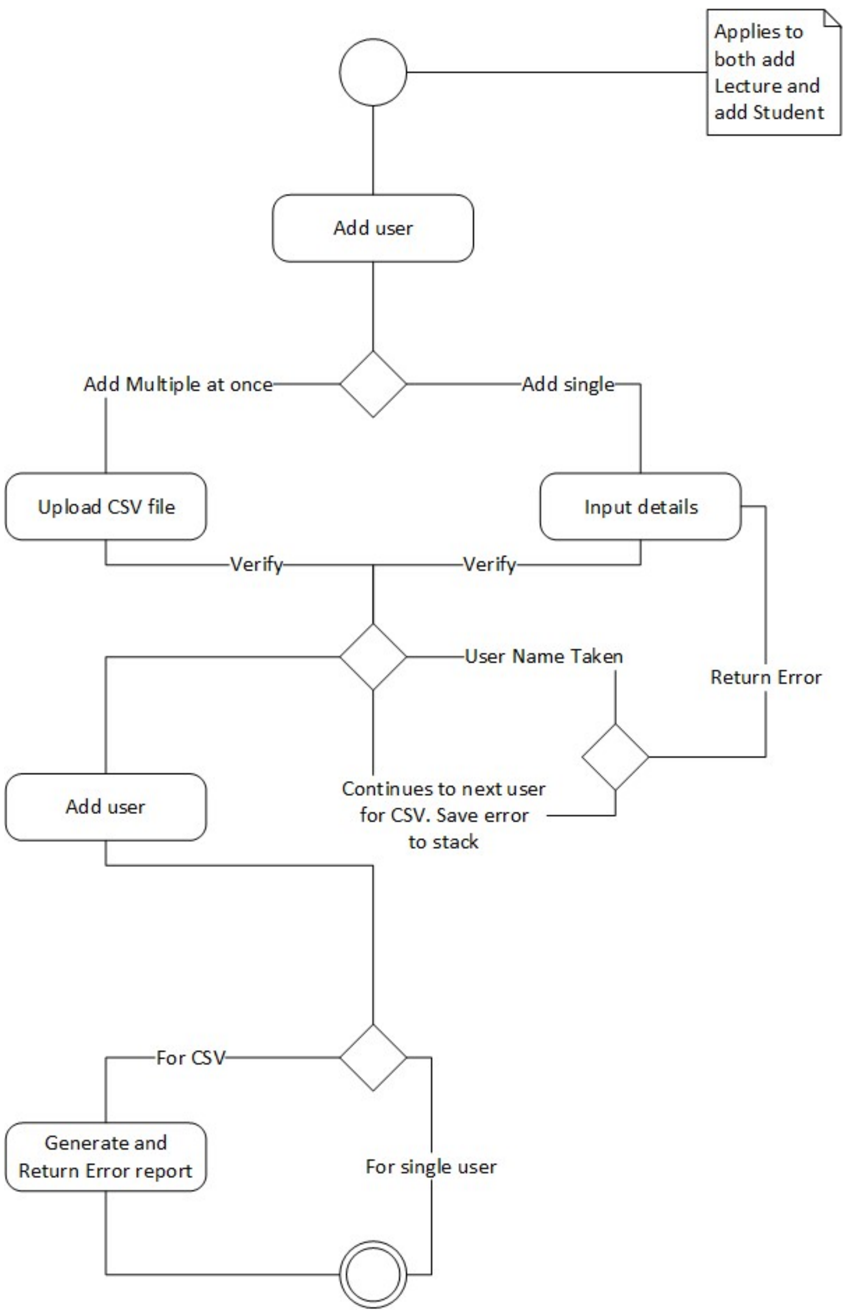

Figure 4 Add User Activity Diagram

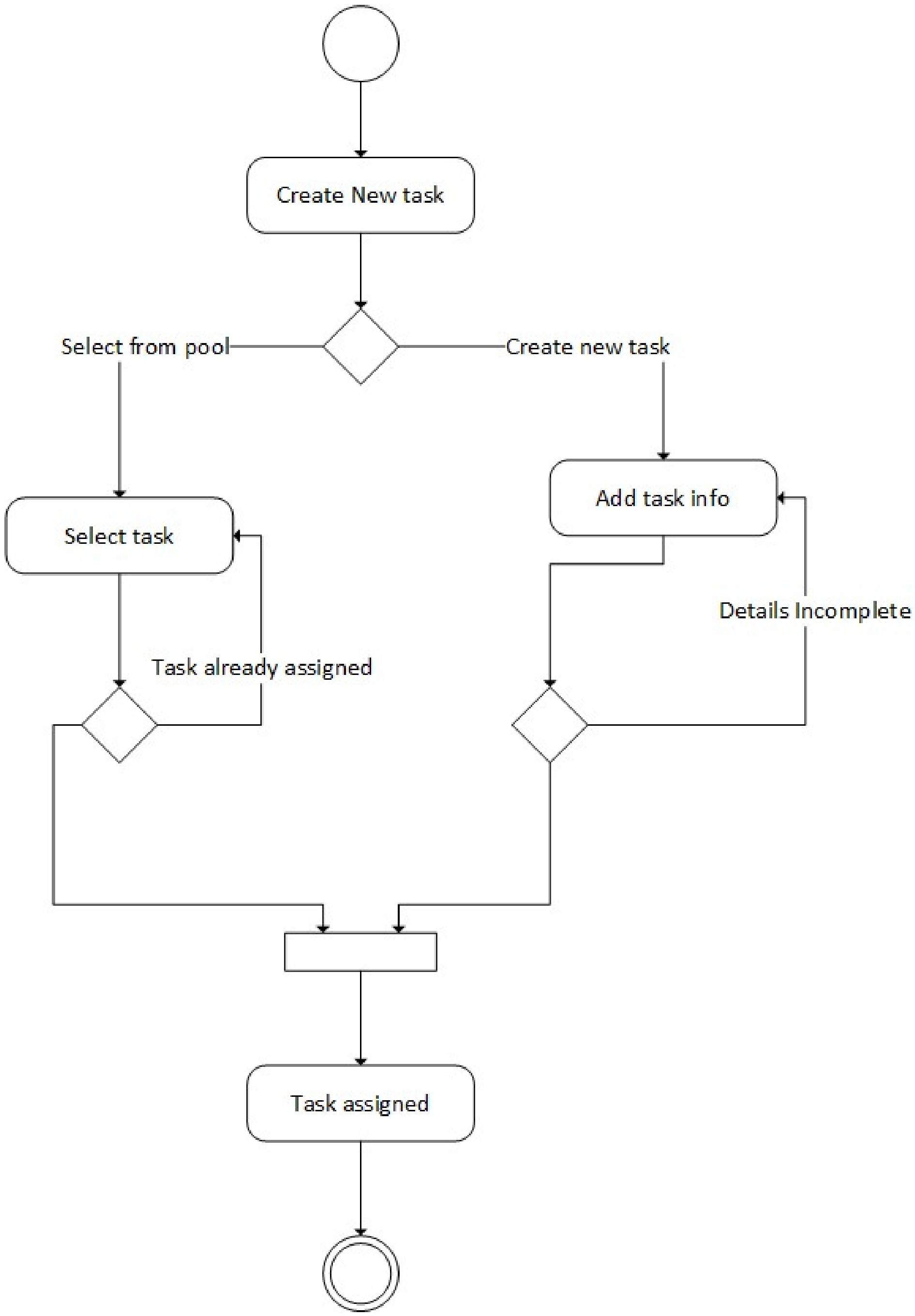

Figure 5 Assign Task Activity Diagram

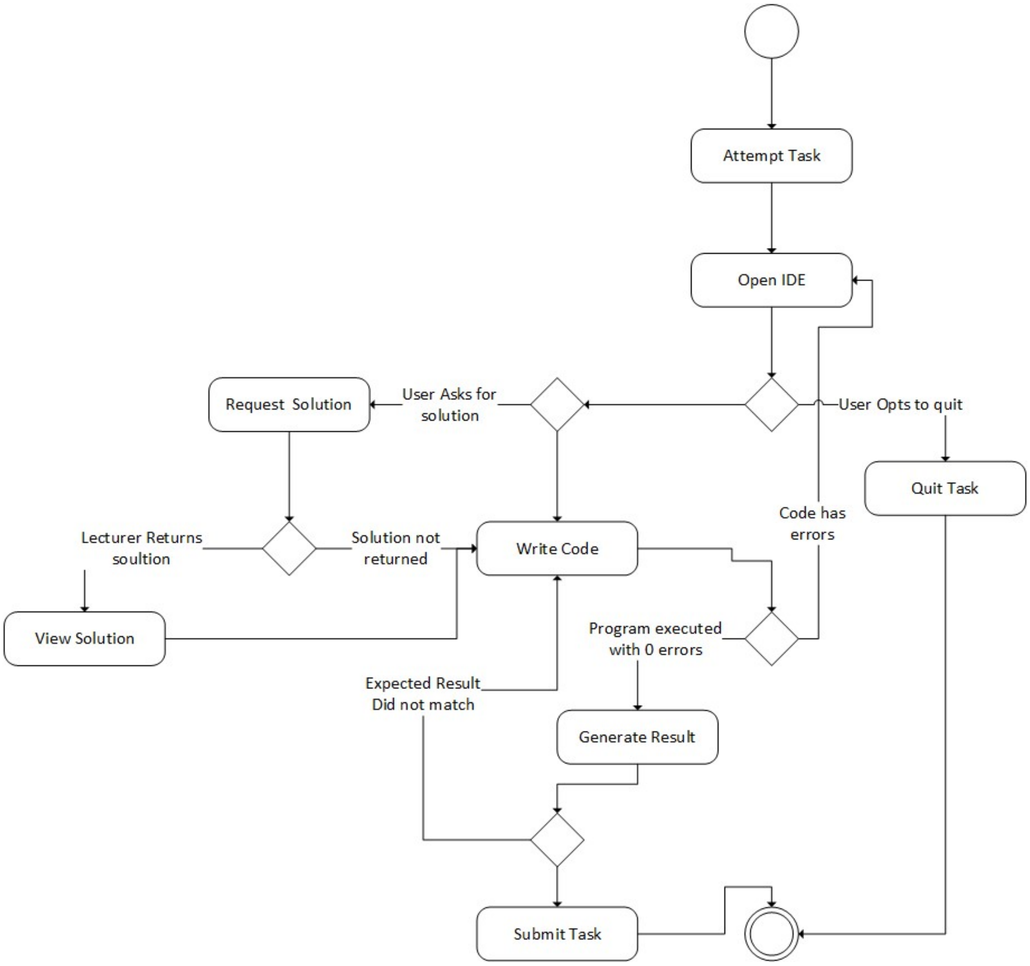

Figure 6 Attempt Task Activity Diagram

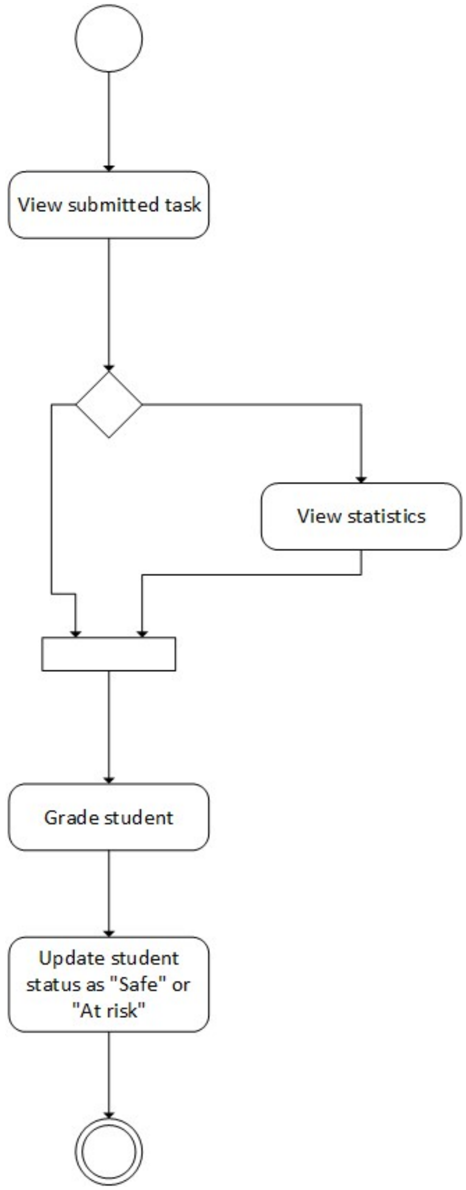

Figure 7 Grade Student Activity Diagram

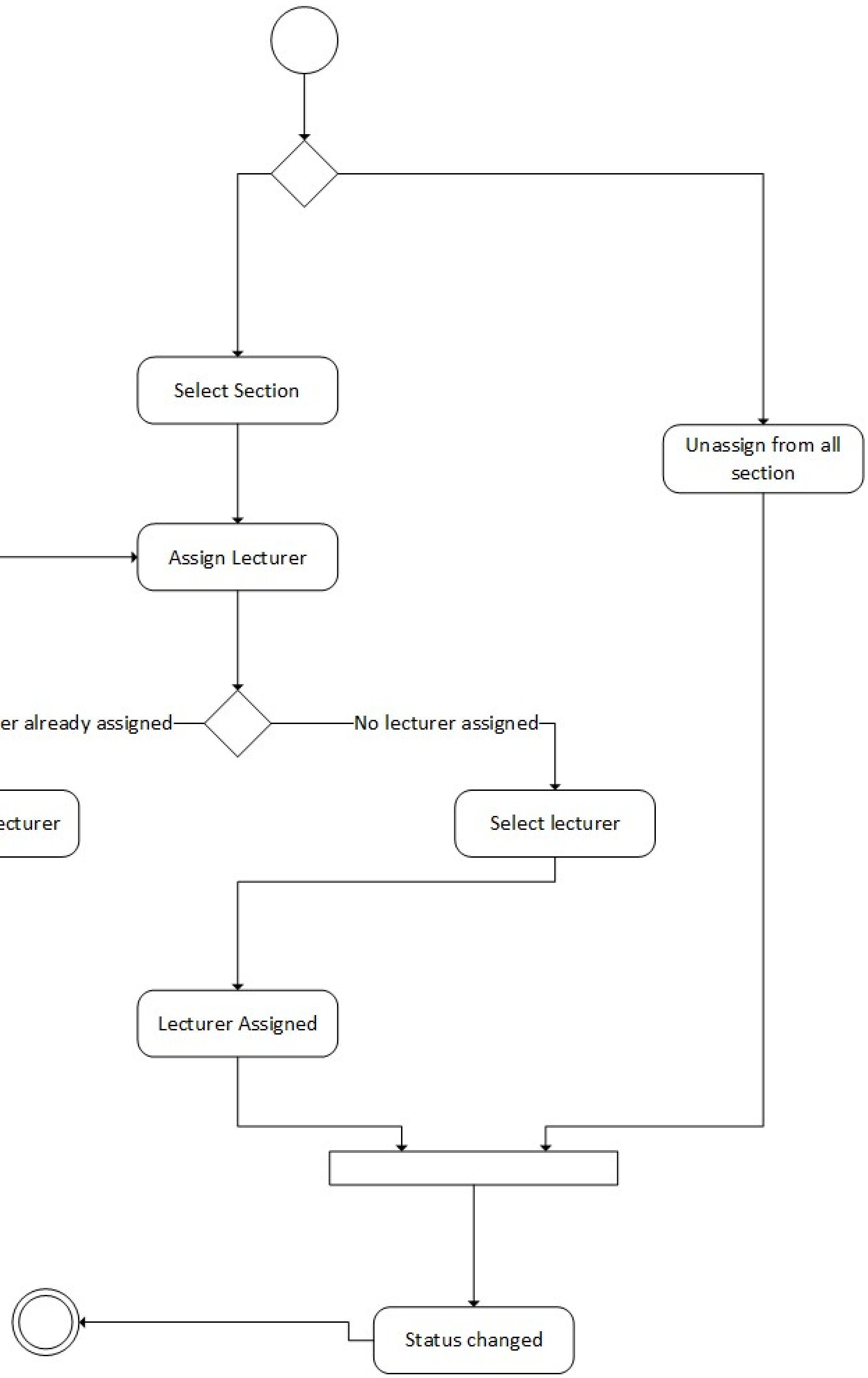

Figure 8 Lecturer Status Activity Diagram

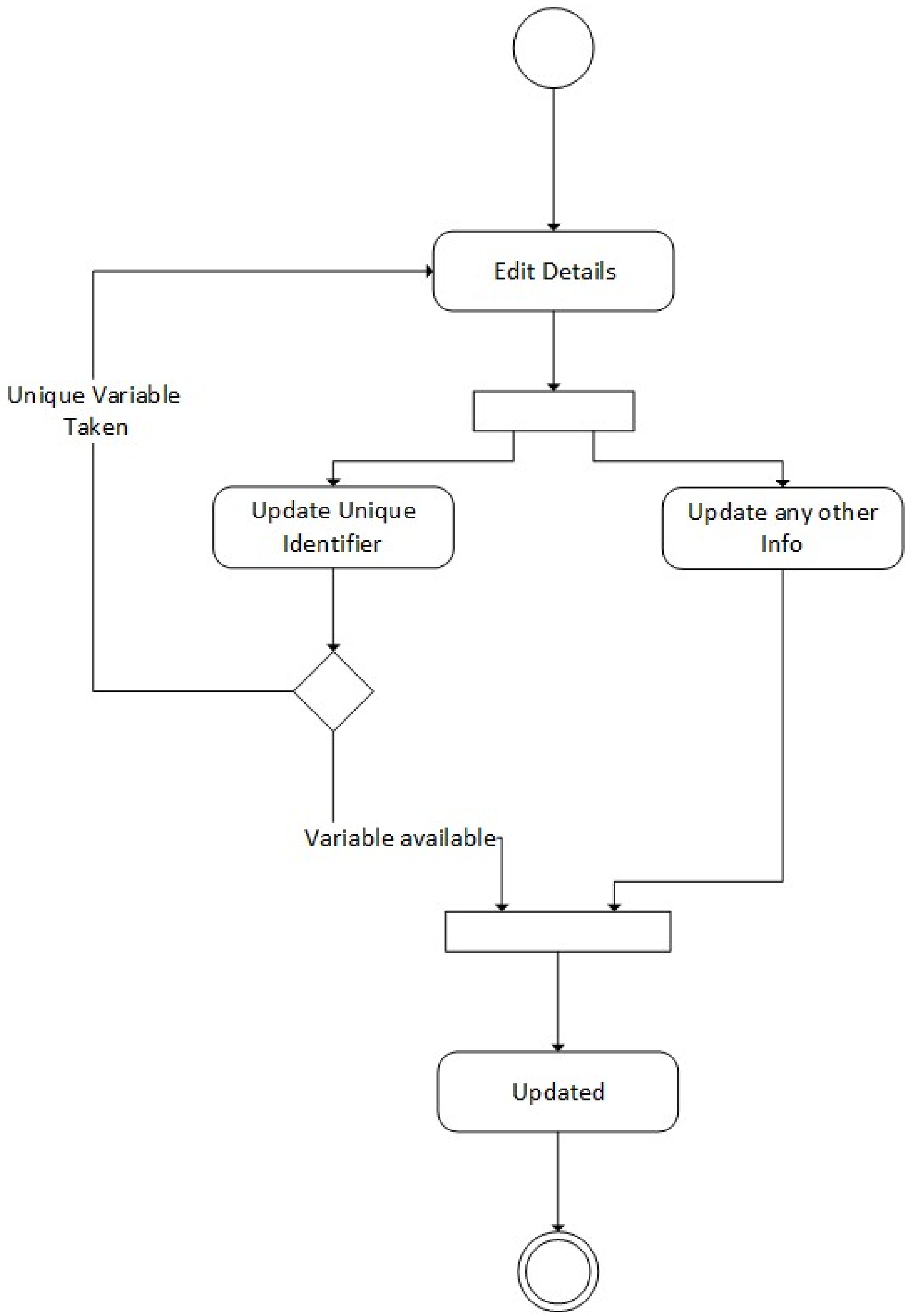

Figure 9 Update User Activity Diagram

4.4 Design Models

Following diagrams discuss design model for Student Assessment Platform:

4.4.1 Structural Diagrams

4.4.1.1 Class diagram

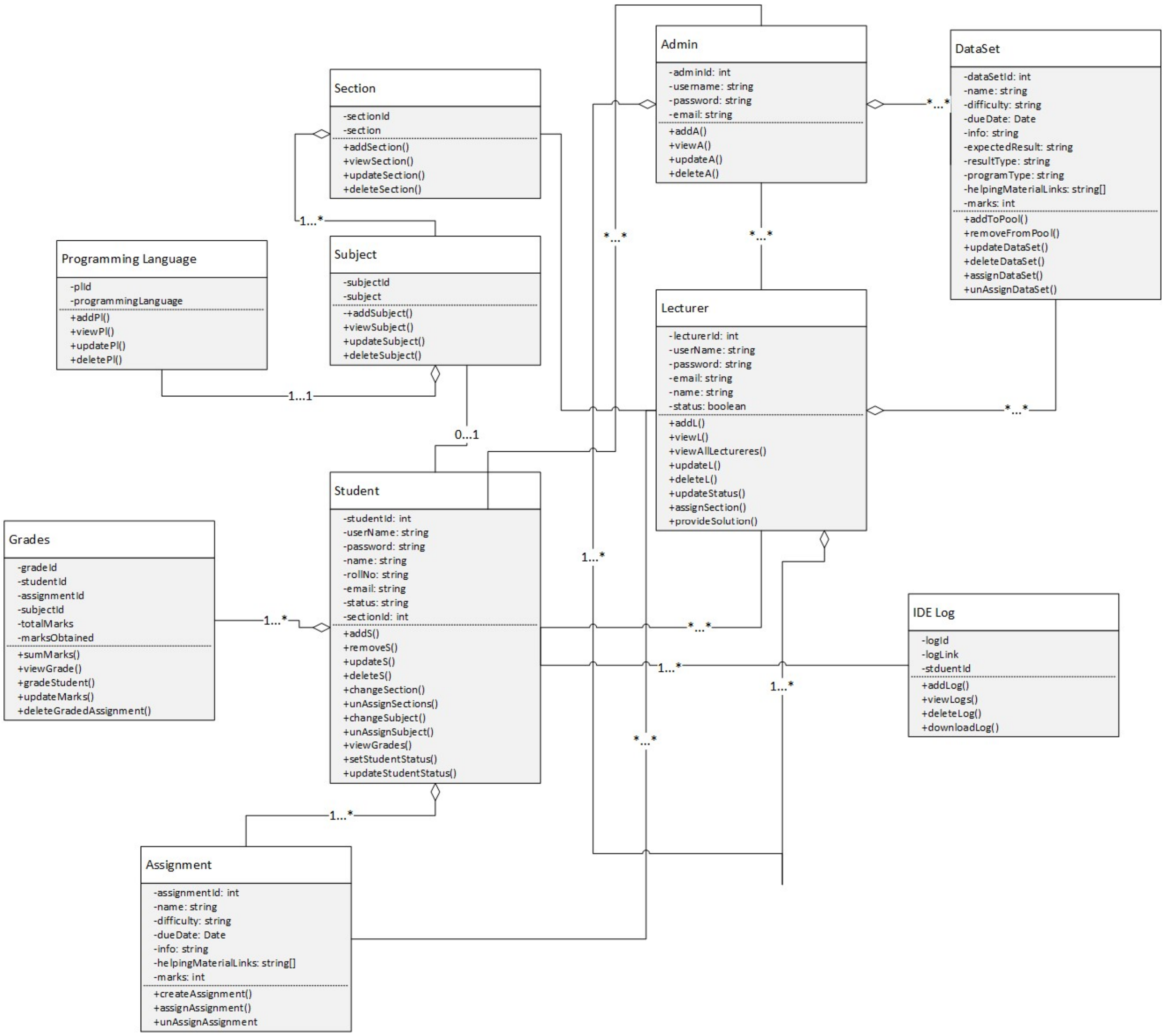

Figure 10 Class Diagram

Following are the classes for Student Assessment Platform:

Student Class, Admin Class, Lecturer Class, Section Class, Subject Class, Programming Language Class, Grades Class, Assignment Class, Mail Class, Notification Class, IDE Log Class, Dataset Class

4.4.2 Behavioral Diagrams

4.4.2.1 Sequence diagram

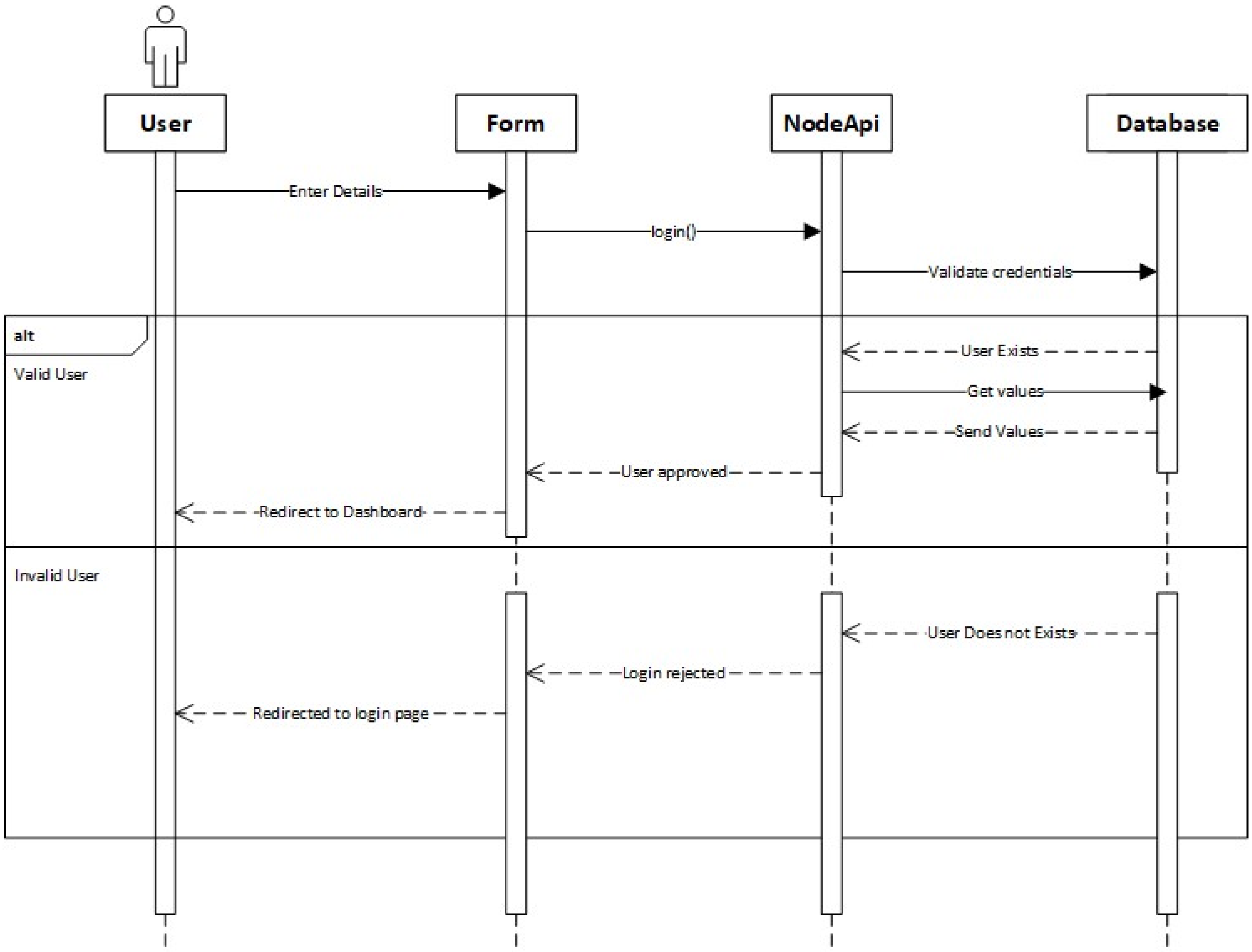

Figure 11 Login Sequence Diagram

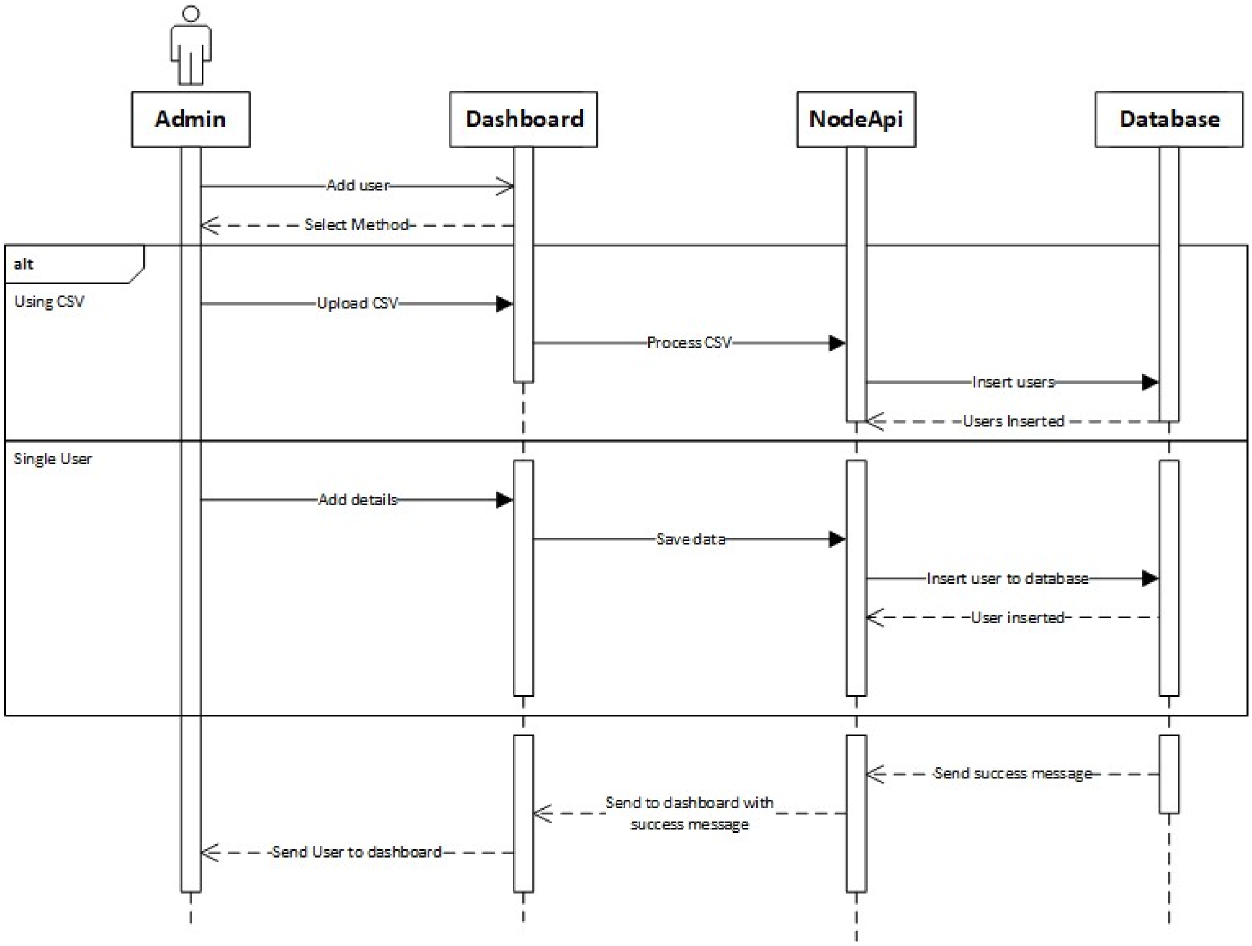

Figure 12 Add User Sequence Diagram

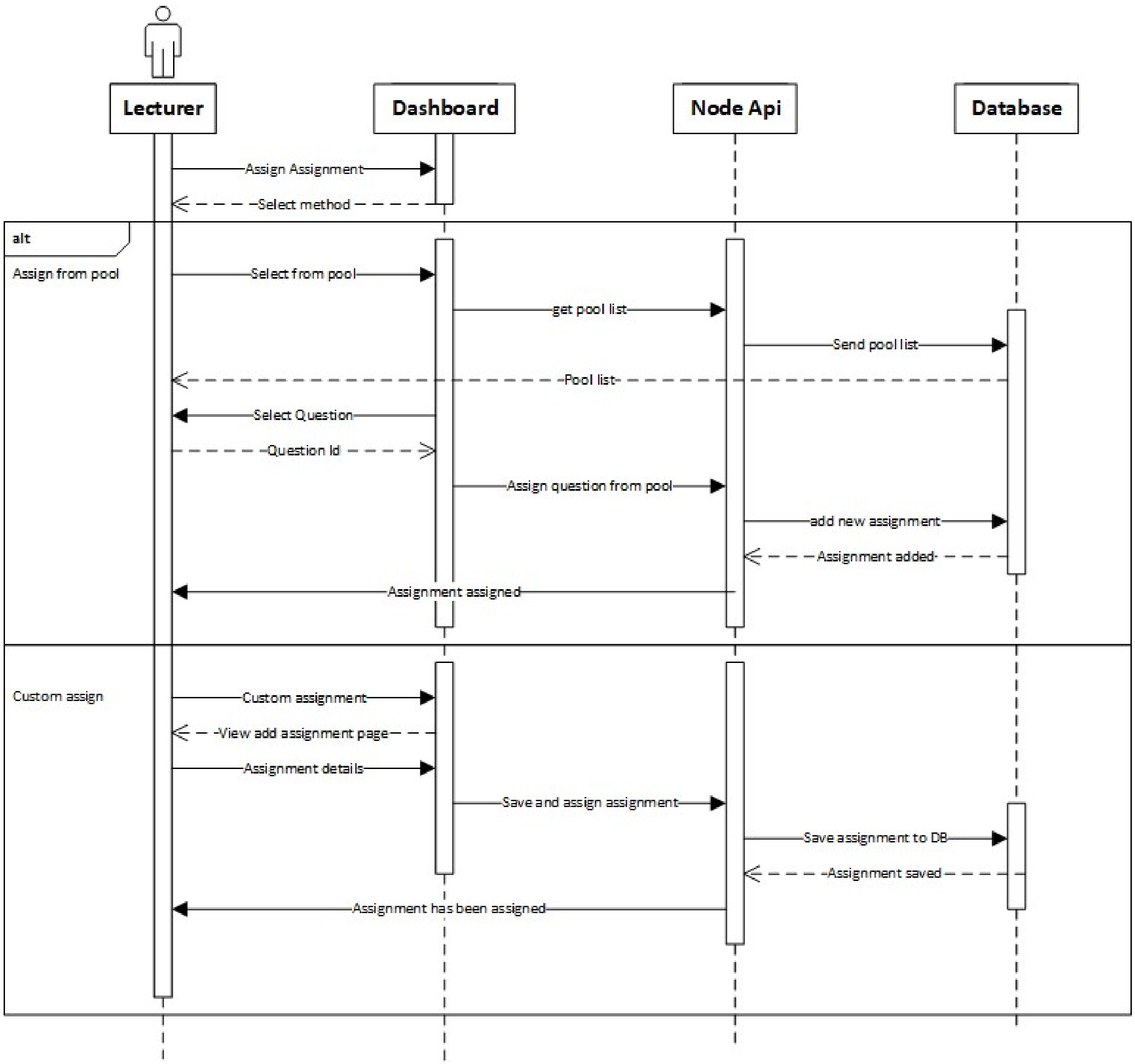

Figure 13 Assign Assignment Diagram

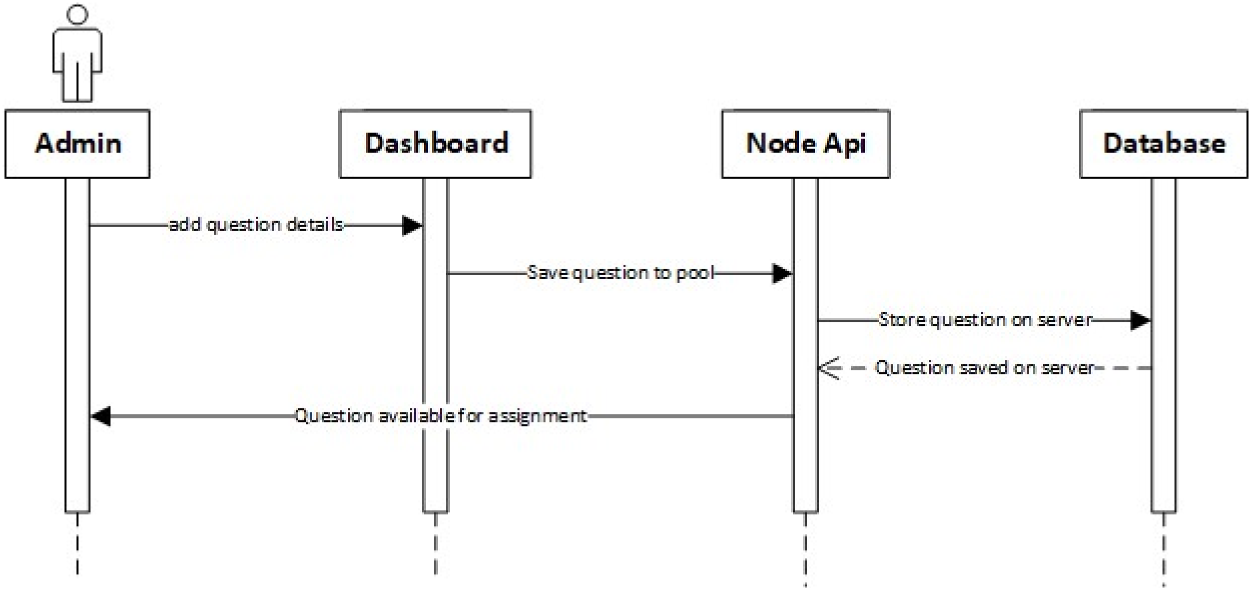

Figure 14 Adding Questions Sequence

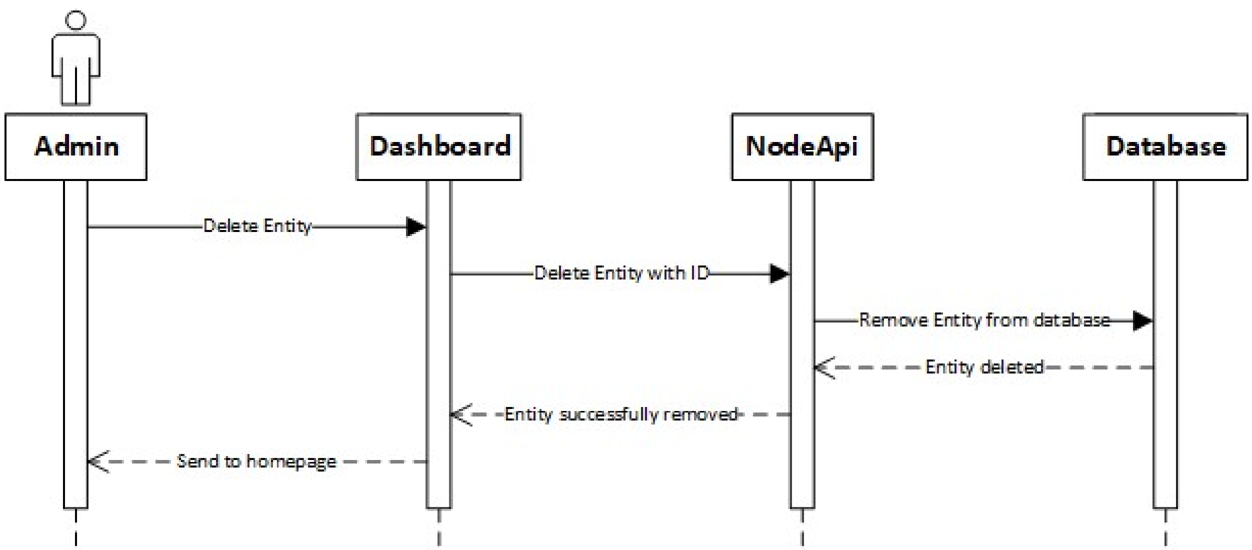

Figure 15 Delete Entity Diagram

5 Implementation

5.1 Algorithm

Notification algorithm:

This algorithm will notify students about the assignments that are assigned or pending. On time notifications are important so that assignments are not missed. Also, lecturer will get the notification when any student is at risk or if the entire class has submitted the task.

At Risk Students:
The IDE will be powerful enough to log and track each student's performance and the live report will be given to the lecturer, if any student is at risk. The lecturer will be informed as well.

5.2 External APIs

- Judge0: It will be used to compile the student's code and return the compiled output.
- RestAPI: Custom RestAPIs will be used to handle various tasks like session management and user logging.
- WYSIWYG Editor: Used for indentation and tweaking the text so that the teacher can deliver assignments in a better way.

5.3 User Interface

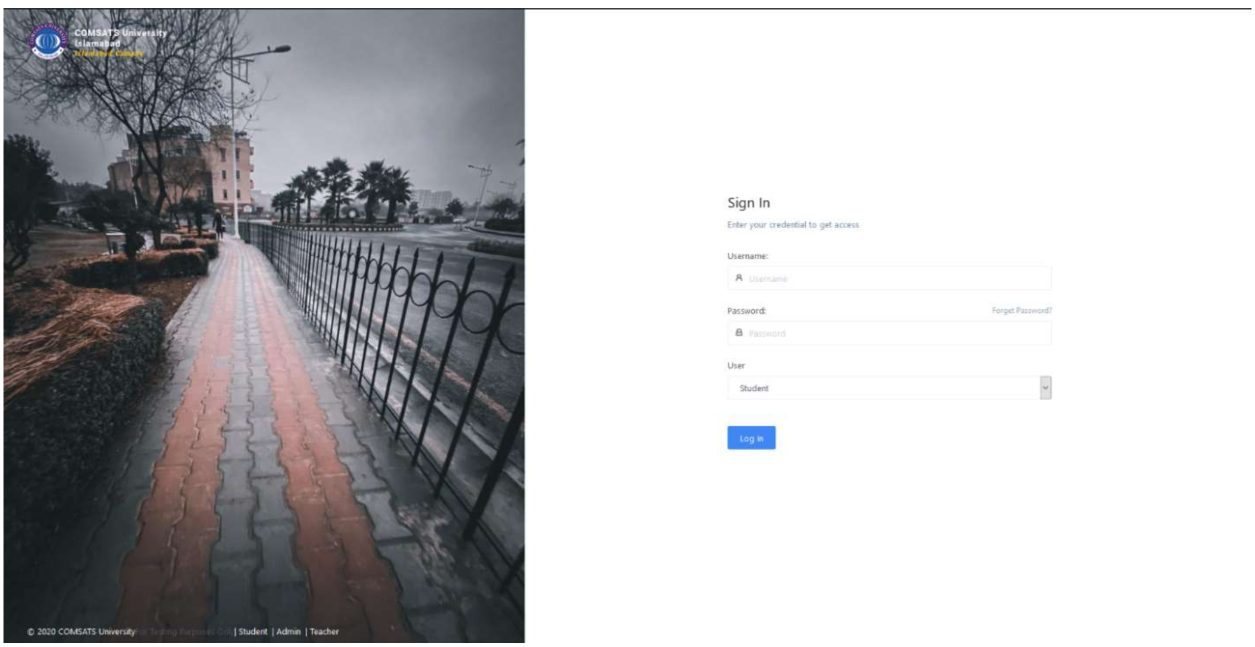

Figure 16 Login Page

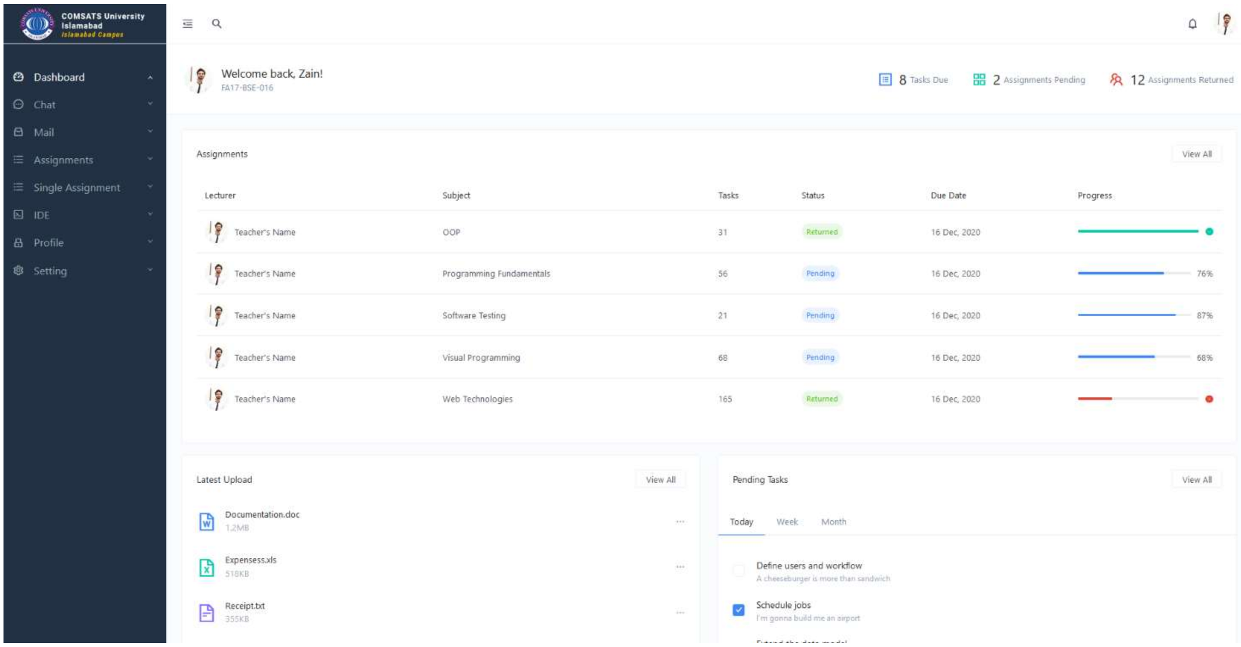

Figure 17 Student Dashboard

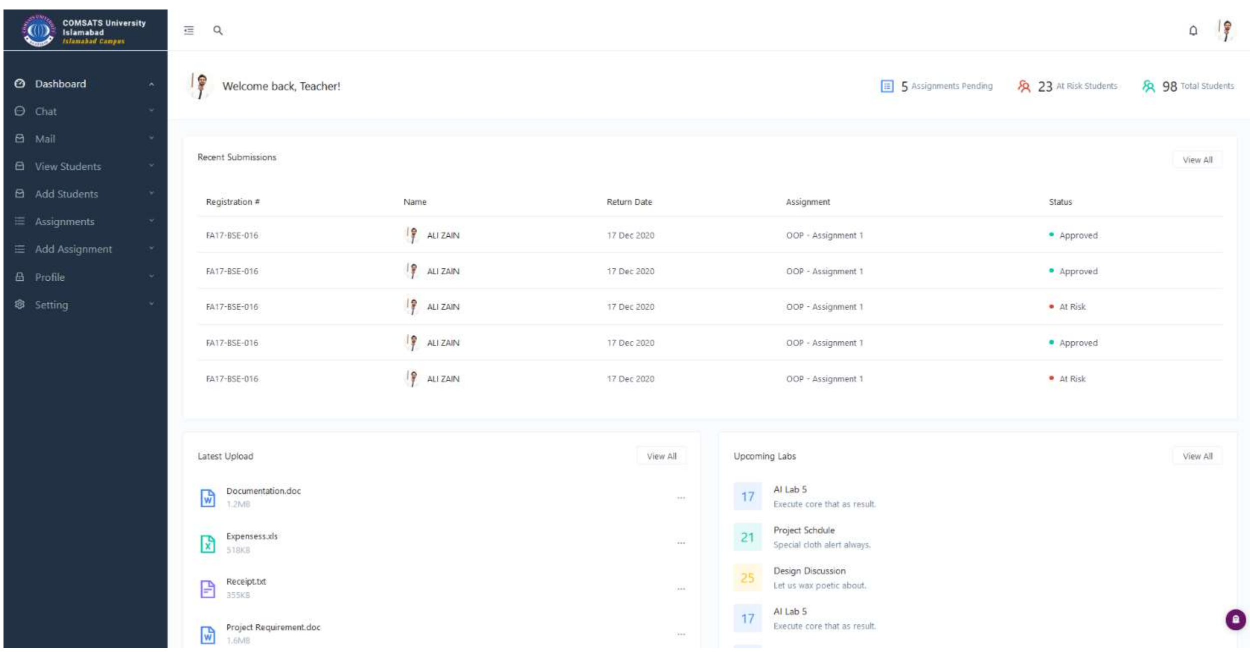

Figure 18 Lecturer Dashboard

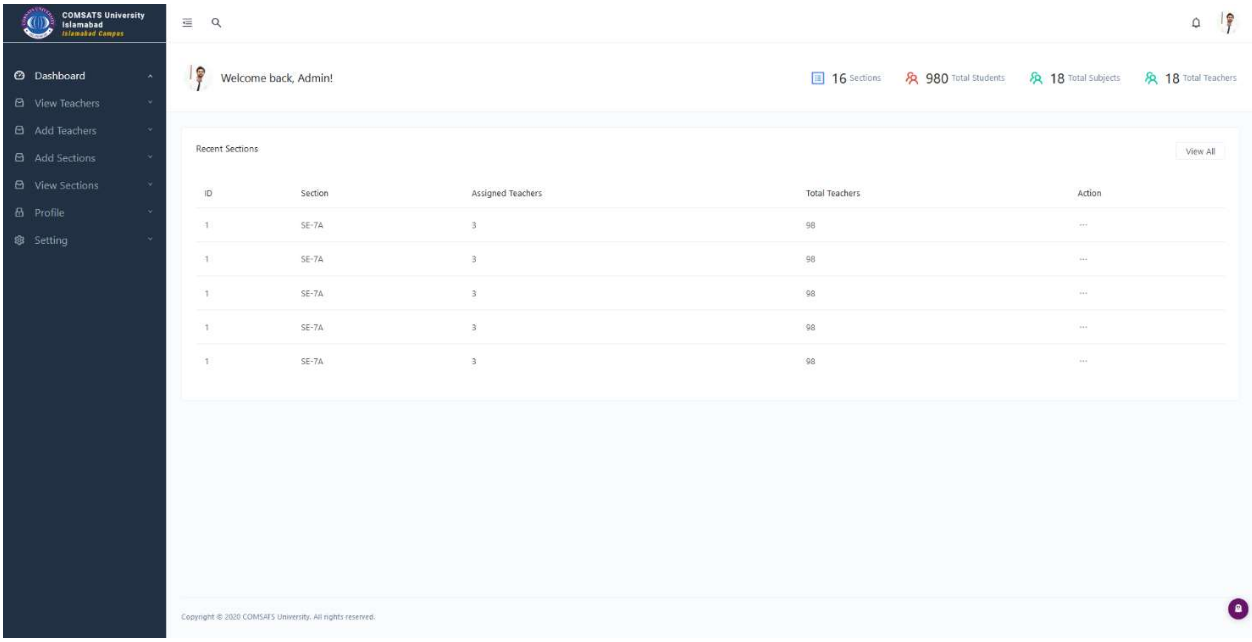

Figure 19 Admin Dashboard

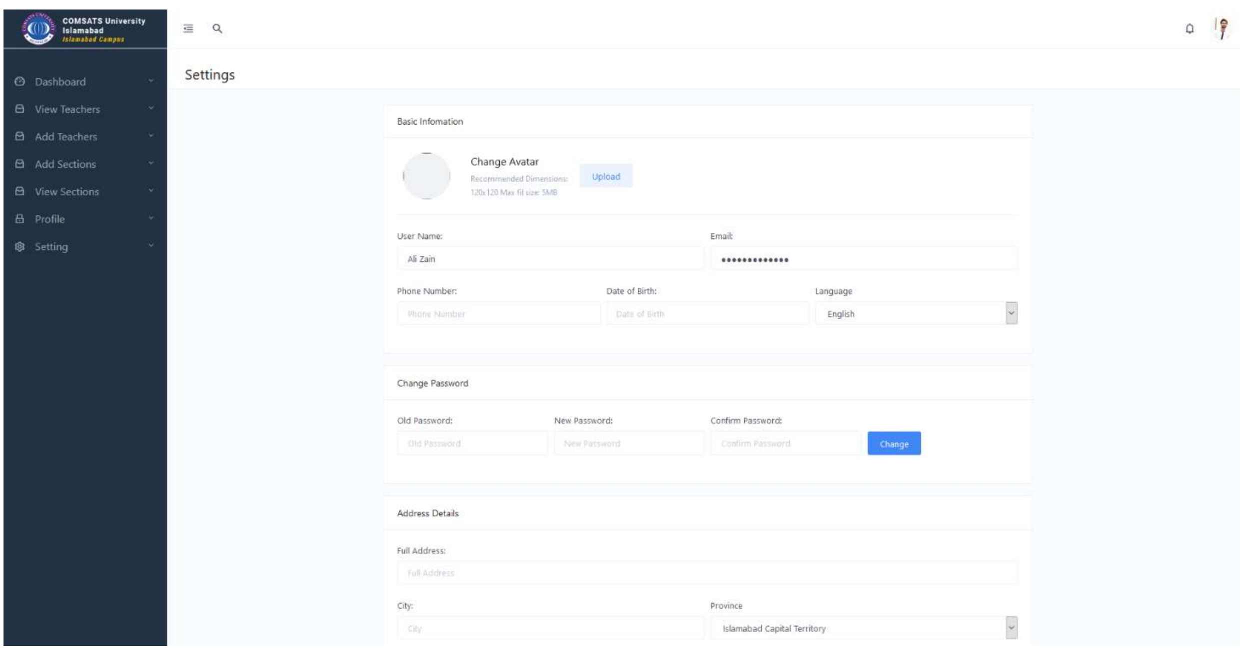

Figure 20 Settings Page

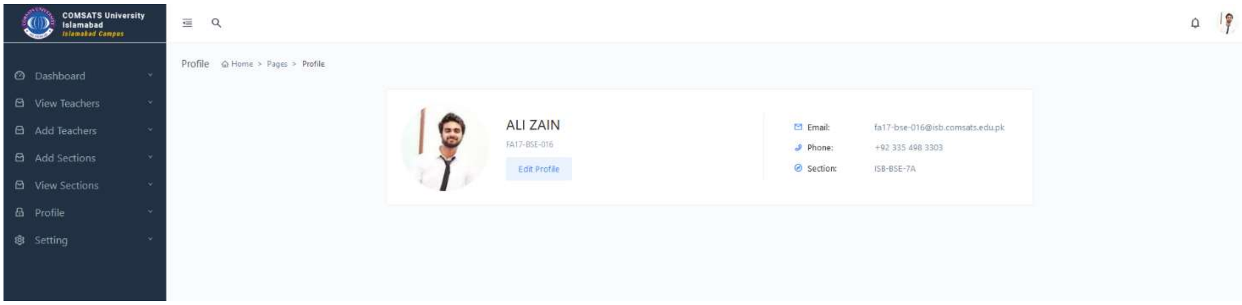

Figure 21 Profile Page

Figure 22 Dropdown Section

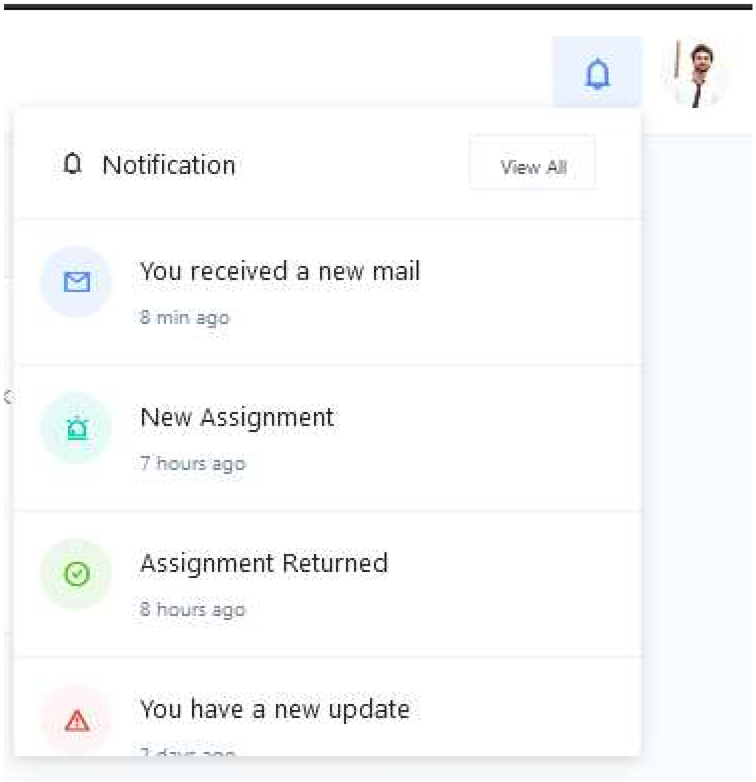

Figure 23 Notification Dropdown

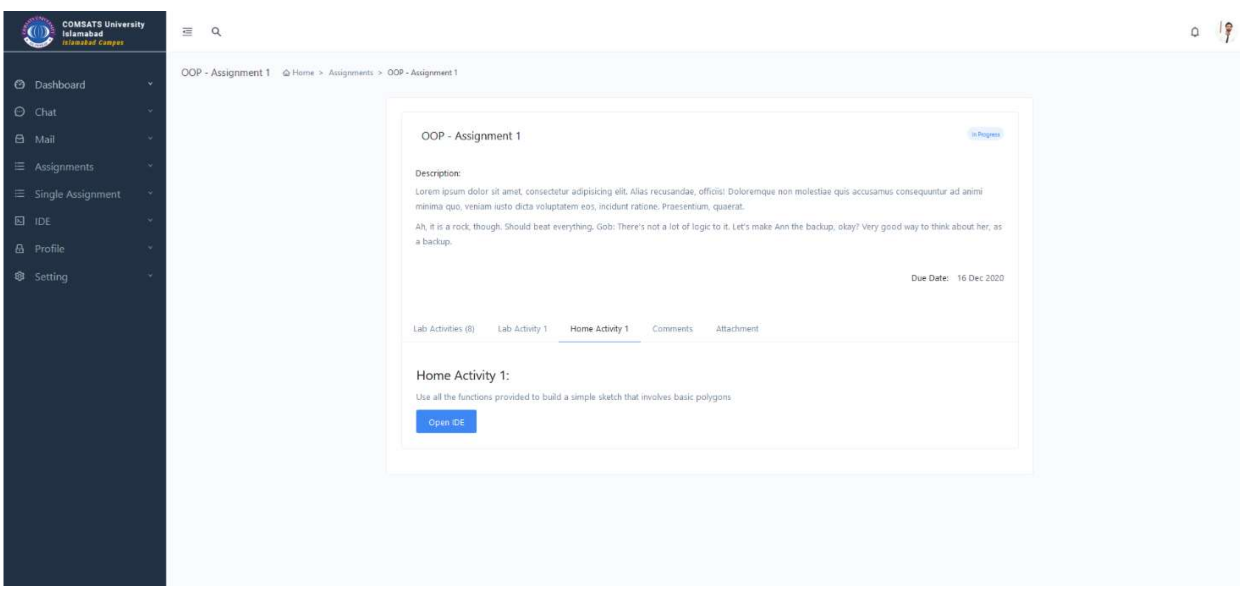

Figure 24 Assignment Page – Student

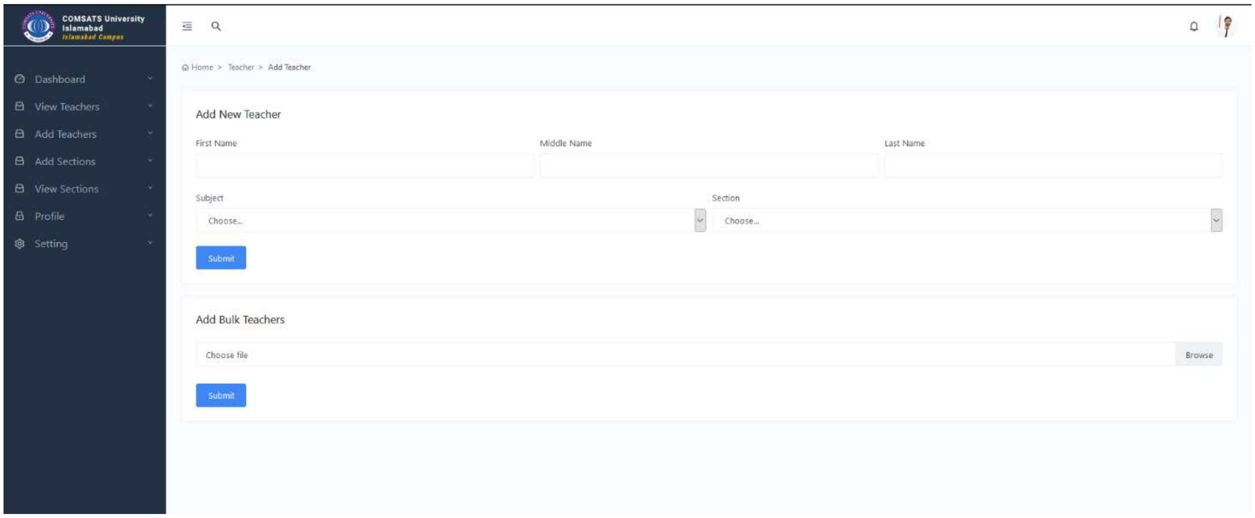

Figure 25 View All Teachers – Admin

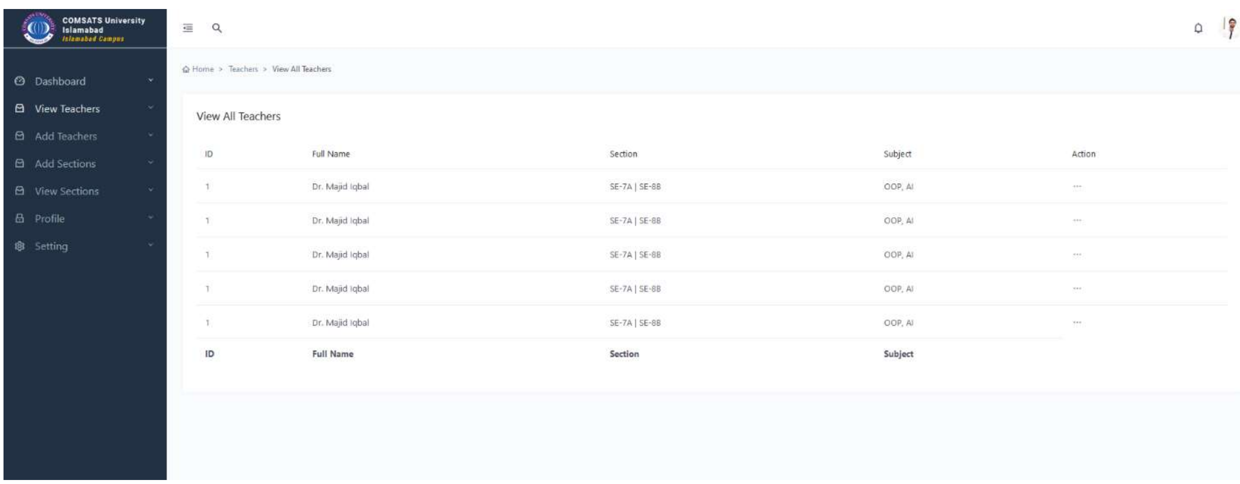

Figure 26 Add Teacher Page – Admin

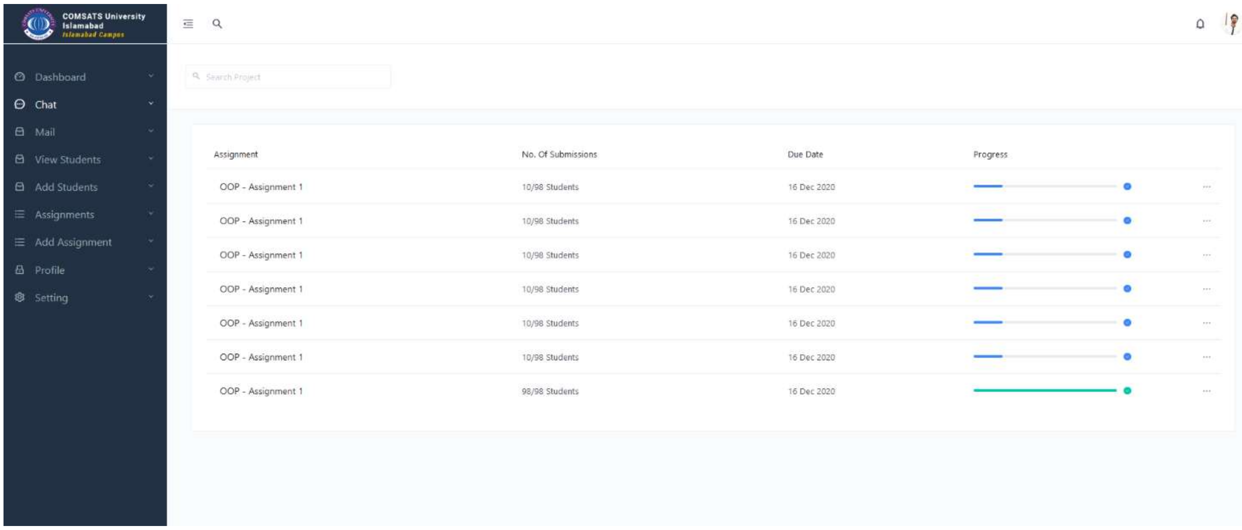

Figure 27 View Assignments - Teacher

6 Testing and Evaluation

6.1 Unit Testing

Unit Testing 1: Login as Student

Testing Objective: To ensure the login form is working correctly.

Test Case Id: UT_001

Test Case Description: Test the login functionality.

Test Scenario: Verify on entering valid username and password, the user can login.

Table 6.1.1: Test Cases for Login

No.	Test Case/Test Script	Test Data	Expected Result	Actual Result	Pass/Fail/Not Executed/ Suspended
1.	Verify user field can handle entry of any length and type.	Make long and short alpha-numeric entries	User input visible in text field as entered	As Expected,	Pass
2.	Verify password field can handle entry of any length and type.	Make long and short alpha-numeric entries	Entry keyed in without an issue	As Expected,	Pass
3.	Verify password field	Key in any set of	Entry to the	As	Pass

	masks entry	characters	password field is masked	Expected,	
4.	Verify login button triggers authentication sequence	Fill in the login fields and click the sign in button	User is authenticated	As Expected,	Pass
5.	Verify credentials verification is functional	Key in a correct set of username and password	User is taken to the IDE main dashboard	As Expected,	Pass
6.	Verify credentials verification is functional	Key in an incorrect set of username and password	A dialog pops up to inform the user about the bad authentication	As Expected,	Pass

Unit Testing 2: Navigate Dashboard

Testing Objective: To ensure the user can navigate the dashboard.

Test Case Id: UT_002

Test Case Description: Test the Dashboard functionality.

Test Scenario: Verify user can navigate the dashboard as intended.

Table 6.1.2: Test Cases for Dashboard Navigation

No.	Test Case/Test Script	Test Data	Expected Result	Actual Result	Pass/Fail/Not Executed/ Suspended
1.	Verify Dashboard is only visible if the user is authenticated	Access dashboard without authenticate	Dashboard asks user to login	As Expected,	Pass
2.	Verify Dashboard is only visible if the user is authenticated	Access dashboard while authenticated	Dashboard comes into view	As Expected,	Pass
3.	Verify Dashboard shows information pertaining to the user currently authenticated	Access dashboard while authenticated	Dashboard shows data for the currently authenticated user	As Expected,	Pass
4.	Verify user can navigate to bundled resources via the provided controls.	Click on the resources button	Resource's tab comes into view	As Expected,	Pass

Unit Testing 3: Verify Smart IDE

Testing Objective: To ensure the IDE allows to write code and linting it as expected.

Test Case Id: UT_003

Test Case Description: Test code writing and linting functionality

Test Scenario: Verify that code can be written and linting in the code editor.

Table 6.1.3: Test Cases for IDE

No.	Test Case/Test Script	Test Data	Expected Result	Actual Result	Pass/Fail/Not Executed/ Suspended
1.	Verify code can be written in the editor.	Key in print (‘hello world’)	Code written in the code editor	As Expected,	Pass
2.	Verify code linting is working as expected.	Key in print(a)	Code editor reports error as undefined name ‘a’	As Expected,	Pass
3.	Verify code compilation is worked as expected	Key in print (‘hello world’) and press the compile button	The bundled console prints ‘Hello world’	As Expected,	Pass
4.	Verify code debugger is working as expected	Place a break point in the editor and run the debugger	Program halts execution at the selected breakpoint	As Expected,	Pass
5.	Verify Git controls are working as expected	Click the source control version in the side navigation pane	Git repository is initialized in the current folder	As Expected,	Pass
6.	Verify code submission and testing works as expected	Current editor code	Code submits and displays the submission message	As Expected,	Pass

6.2 Functional Testing

The functional testing will take place after the unit testing. In this functional testing, the functionality of each of the module is tested. This is to ensure that the system produced meets the specifications and requirements.

Functional Testing 1: Login with provided credentials.

Test Objective: To ensure that the correct page with the correct navigation bar is loaded.

Test Case Id: FT_001

Test Case Description:

Test Scenario:

Table 6.2.1: Test Cases for Login

No.	Test Case/Test Script	Test Data	Expected Result	Actual Result	Pass/Fail/Not Executed/ Suspended
1.	Verify user login after click on the 'Login' button on login form with correct input data.	Username: test Password: 123456	Successfully log into the main page of the system.	As Expected,	Pass
2.	Verify user login after click on the 'Login' button on login form with incorrect input data.	Username: test Password: 1234	Display an error dialog citing incorrect credentials entry	As Expected,	Pass

Functional Testing 2: Verify open file.

Testing Objective: To ensure the file is opening as expected.

Test Case Id: FT_002

Test Case Description: Test file opening functionality

Test Scenario: Verify that files can be opened in the code editor.

Table 6.2.2: Test Cases for Open File

No.	Test Case/Test Script	Test Data	Expected Result	Actual Result	Pass/Fail/Not Executed/ Suspended
1.	Verify file opens in the code editor.	Open a file using the menu bar open file option	A file opens in the currently opened project folder	As Expected,	Pass

2.	Verify file is opened in editor via the keyboard shortcut.	Create a file using the keyboard short cut Ctrl + o	A file opens in the currently opened project folder	As Expected,	Pass

Functional Testing 3: Verify code compilation.

Testing Objective: To ensure the code compilation is working as expected.

Test Case Id: FT_003

Test Case Description: Test compilation functionality

Test Scenario: Verify that code compilation takes place when the compile button is pressed.

Table 6.2.3: Test Cases for Code Compilation

No.	Test Case/Test Script	Test Data	Expected Result	Actual Result	Pass/Fail/Not Executed/ Suspended
1.	Verify code compiles on button press.	Code	Code compiles to show the output in the console	As Expected,	Pass
2.	Verify exceptions are shown on program crash.	Erroneous code	Exception is shown in the console after a program crash	As Expected,	Pass

Functional Testing 4: Verify access to assignments.

Testing Objective: To ensure the access to assignments is working as expected.

Test Case Id: FT_004

Test Case Description: Test assignments functionality

Test Scenario: Verify that assignments are showing up and load.

Table 6.2.4: Test Cases for Assignments

No.	Test Case/Test Script	Test Data	Expected Result	Actual Result	Pass/Fail/Not Executed/ Suspended
1.	Verify assignments pane shows up when view assignments button is pressed on user dashboard.	Assignments	Assignment's pane comes into view with all the assignments listed	As Expected,	Pass
2.	Verify a selected	Assignment	Assignment project	As	Pass

	assignment loads in the code editor.		opens in the editor with the code editor and instructions side-by-side	Expected,	

Functional Testing 5: Verify search term.

Testing Objective: To ensure search term is working as expected.

Test Case Id: FT_005

Test Case Description: Test search term functionality

Test Scenario: Verify that term searches within the editor work.

Table 6.2.5: Test Cases for Search term

No.	Test Case/Test Script	Test Data	Expected Result	Actual Result	Pass/Fail/Not Executed/ Suspended
1.	Verify term search is working when an item present in the opened file is searched for.	Search item: java	Term ‘print’ is highlighted in the editor	As Expected,	Pass
2.	Verify term search is working when a search term does not present in the file is keyed in.	Search item: cpp	Search bar shows zero finds	As Expected,	Pass

Functional Testing 6: Verify rename file.

Testing Objective: To ensure rename file is working as expected.

Test Case Id: FT_006

Test Case Description: Test rename file functionality.

Test Scenario: Verify that rename file changes the name of a file in the current project.

Table 6.2.6: Test Cases for Rename File

No.	Test Case/Test Script	Test Data	Expected Result	Actual Result	Pass/Fail/Not Executed/ Suspended
1.	Verify rename files changes the name of the selected file when rename option is selected from the context menu activated by right clicking on the desired	Name: updated.py	File name is updated with the provided name	As Expected,	Pass

	file.				
2.	Verify rename files does not change the name of the selected file when rename option is selected from the context menu activated by right clicking on the desired file and the file name already exists in the directory.	Name: main.py	File name remains unchanged	As Expected,	Pass

6.3 System Testing

The system testing showed us that the defined project was developed with respect to the requirements and functionalities stated in Scope, SRS and SDD Document. It is as per our and our supervisor's expectation and stands completed with minor revisions.

7 Conclusion and Future Work

7.1 Conclusion

This dashboard is a complete application designed as a Web Application to run on any hardware and software platform without compromising UX or functionality. It integrates all the features which are required for lecturers/faculty to monitor their students all together on one platform. It has separate accounts for both students and lecturers which ensures they perform specialized functionality as well. Being responsive in nature ensures that cross-platform usage will be promoted, and all application instances will perform without issues.

7.2 Future Work

We believe that the project has came to its completion, nevertheless there is some minor work required on the IDE to make it even better than it is right now. Apart from that the design can be tweaked in some places to better match the overall feel of the website.

8 References

C.D HUNDHAUSEN, D. O. (2017). *IDE-Based Learning Analytics for Computing Education:*

Carlos Fernández Medina, J. R. (2013). *Assistance in computer programming learning using educational data mining.*

codecademy. (2011). Retrieved from codecademy: https://www.codecademy.com/

Codewars. (2012). Retrieved from Codewars: https://dev.codewars.com/

David Azcona, I.-H. H. (2018). *PredictCS: Personalizing Programming Learning by Leveraging Learning Analytics.*

HackerRank. (2009). Retrieved from HackerRank: https://www.hackerrank.com

hermanzdosilovic. (2016). Retrieved from Judge0 - API: https://github.com/judge0/api

Lu Owen H.T, H. J. (2017). *Applying learning analytics for improving student's engagement and learning outcomes in an MOOCs enabled collaborative programming course.* Routledge.

OCR: For guidance, please follow the link: https://moov.ai/en/blog/optical-character-recognition-ocr/

Recipes API database: For guidance, please follow the link: https://github.com/tabatkins/recipe-db/blob/master/db-recipes.json

Ingredients Dataset: For guidance, please follow the link: https://www.kaggle.com/c/whats-cooking/data?select=sample_submission.csv.zip